

Job Matcher Project

By

Ong Weng Kai



FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF
MANAGEMENT AND TECHNOLOGY
KUALA LUMPUR

ACADEMIC YEAR
2024/25

Job Matcher Project

By

Ong Weng Kai

Supervisor: Fatin Izzati Binti Ramli

A project report submitted to the
Faculty of Computing and Information Technology
in partial fulfillment of the requirement for the
Bachelor of Computer Science (Honours)

Department of Mathematical and Data Science
Faculty of Computing and Information Technology
Tunku Abdul Rahman University of Management and Technology
Kuala Lumpur

Copyright by Tunku Abdul Rahman University of Management and Technology.

All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.

Student Name Ong Weng Kai

Bachelor of Computer Science (Honours) in Data Science

ID:2203309

Abstract

The project aims to revolutionize the job recruitment process by introducing an AI-driven system that enhances how job seekers are matched with suitable opportunities. It addresses key recruitment challenges such as managing high application volumes and unclear job descriptions, which often lead to inefficiencies and increased hiring costs. By leveraging natural language processing techniques like cosine similarity and TF-IDF vectorization, the system accurately aligns resumes with job listings, streamlining the screening process and minimizing mismatches.

The scope of this project encompasses developing a comprehensive platform that includes features for resume uploading, preprocessing, and an employer interface for robust job management. Both job seekers and employers benefit from functionalities such as advanced filtering, job ranking by relevance, and secure logins for managing postings and applicant data.

Methodologically, the system is built using Python with Streamlit for the interface and MongoDB for backend data storage, supporting structured storage and retrieval of job and user data. The architecture is designed to process data in real-time, ensuring that job listings and applicant matches are consistently updated.

Comprehensive testing covered various aspects including functionality, usability, and security. Results affirmed the system's effectiveness in simplifying recruitment processes, offering timely and precise job matches. Nevertheless, the system's current version does not integrate with external job boards and could be improved by incorporating real-time data feeds for a more extensive listing of opportunities.

In conclusion, the project underscores the potential of AI in recruitment, significantly reducing hiring timelines and costs. Future enhancements could focus on expanding system integration, increasing scalability, and implementing adaptive AI models to better meet changing market demands.

Acknowledgement

This section extends heartfelt gratitude to those who have contributed significantly to the fruition of this project. Special thanks are owed to the project supervisor, Fatin Izzati Binti Ramli, whose guidance was invaluable. Appreciation is also extended to family members for their unwavering support and encouragement, as well as to colleagues and peers who provided insights and assistance throughout the project's development. Their collective contributions were crucial in navigating the challenges faced during the project and greatly aided in its successful completion.

1 Table of Contents

Declaration.....	ii
Abstract.....	iv
Acknowledgement.....	iv
1 Introduction.....	9
1.1 Background.....	9
1.2 Problem Statement.....	10
1.3 Objective.....	11
1.4 Scope of Study.....	12
1.4.1 Included in the Study:.....	12
1.4.2 Excluded from the Study:.....	13
1.5 Chapter Summary and evaluation.....	14
2 Literature Review.....	18
2.1 Technology involved.....	18
2.1.1 Cosine similarity.....	18
2.1.2 Term Frequency-Inverse Document Frequency (TF-IDF).....	19
2.1.3 Named Entity Recognition (NER).....	21
2.1.4 Support Vector Machine (SVM).....	22
2.1.5 Logistic regression.....	27
2.1.6 Random Forest.....	27
2.2 Similar Project.....	28
3 Methodology and Requirements Analysis.....	32
3.1 Preprocessing.....	32
3.1.1 Data Cleaning.....	32
3.2 Tokenization.....	35
3.2.1 Vectorization.....	36
3.3 Named Entity Recognition (NER) and Cosine Similarity.....	38
3.3.1 Named Entity Recognition (NER).....	38
3.3.2 Cosine similarity.....	40
3.4 Data storage.....	43
3.4.1 Job seeker.....	Error! Bookmark not defined.
3.4.2 Employer.....	43
4 System Design.....	46
4.1 System Flow.....	46
4.1.1 Job Seeker upload their resume (1).....	46
4.1.2 Convert into TXT and read resume(2).....	46
4.1.3 Preprocess the resume and job description(3) (b).....	46
4.1.4 Named Entity Recognition for job description and resume(4) (C).....	47
4.1.5 TF-IDF for resume and job description(5) (D).....	47
4.1.6 Database.....	47
4.1.7 Cosine similarity (ii).....	48
4.2 User Interface.....	49
4.2.1 Employer.....	49
4.2.2 Job seeker.....	51
5 Implementation and Testing.....	56
5.1 Implementation.....	56
5.1.1 Main User Interface.....	56
5.1.2 Main User Interface.....	60

5.1.3	Job Listings	67
5.1.4	Visualization	69
5.2	Testing	71
5.2.1	Main Page test Case	71
5.2.2	Employer Test Case	73
5.2.3	Job Listing Test Case	77
5.2.4	Visualization Interface Test Case	78
5.3	Database and security Configuration	79
5.3.1	MongoDB Usage	79
5.3.2	Security Practices.....	80
5.3.3	Network and Server Settings	80
6	System Deployment.....	82
6.1	Local Deployment Instructions.....	82
7	Discussions and Conclusion.....	84
7.1	Summary.....	84
7.2	Achievements.....	84
7.3	Contributions	85
7.4	Limitations and Future Improvements.....	85
7.5	Issues and Solutions.....	86
	References.....	88
	Appendices.....	92

Table of figure

Figure 1	The formula for cosine similarity	18
Figure 2	Graphic showing vector with similarities close to 1, close to 0 and close to -1	19
Figure 3	Formula for Term Frequency (TF)	20
Figure 4	Inverse Document Frequency (IDF) Formula	20
Figure 5	Formula for TF-IDF	21
Figure 6	Classification VS regression Of SVM.....	23
Figure 7	SVM Hard Margin	24
Figure 8	SVM Soft Margin.....	24
Figure 9	Non-linear SVM.....	25
Figure 10	Non-linear SVM using Kernal Function	26
Figure 11	The equation for linear hyperplane	26
Figure 12	The equation for linear hyperplane	27
Figure 13	simplified workflow of the system.....	32
Figure 14	Flow of How TF-IDF is calculated.	37
Figure 15	Numerical example of TF formula applied.	38
Figure 16	Numerical example of IDF formula applied.....	38
Figure 17	Another Formula of IDF.....	38
Figure 18	Numerical example of TF-IDF formula applied.....	38
Figure 19	Work Flow of NER	39
Figure 20	Example of cosine of the angle between two vectors in a multi-dimensional space.	41
Figure 21	Example of high cosine similarity	41
Figure 22	The Cosine Similarity values for different documents, 1 (same direction), 0 (90 deg.), -1 (opposite directions).	42
Figure 23	Similarity decrease when distance between two vector increase.....	42
Figure 24	Similarity increase when distance between two vector decreases.	42
Figure 25	Cosine similarity Formula	43
Figure 26	System design Work flow.	46
Figure 27	Login, Register page for Employer	49

Figure 28 Employer page after login	50
Figure 29 Page for job seeker to submit resume to the system	51
Figure 30 All the job listing available for Job Seeker.....	52
Figure 31 Visualization Page.....	53

Chapter 1

Introduction

1 Introduction

In this chapter, we explore the development of an AI-driven job-matching system designed to enhance the recruitment process. We begin with the **Background**, which includes the history and current recruitment process. The Next **Problem Statement** identifies the challenges and inefficiencies in current recruitment practices. After that is the **objective**, which includes how the technology is being used to solve the problem. This is followed by the **Scope of the study**, which includes what kind of technology is used and excluded from the study. Finally, there is the **Chapter summary**.

1.1 Background

Recruitment practices have undergone significant transformations over the past few decades, evolving with technological advancements and changes in societal behaviour. In the early 1970s, recruitment was predominantly conducted through traditional methods such as newspaper advertisements and word-of-mouth referrals. (SourceBreaker & SourceBreaker, 2023) This manual process involved physical resumes and in-person interviews, making it time-consuming and geographically limited.

The digital revolution in the 1980s and 1990s marked a pivotal shift with the advent of the internet. Online job boards emerged, allowing employers to post jobs accessible to a broader audience, and candidates could apply for positions electronically. This period saw a significant reduction in hiring timelines and an expansion of the candidate pool beyond local geographies. (*A History of Recruitment - Era 2: The 1980s to 1990* | EMinfo.com, n.d.)

In the early 2000s, the recruitment landscape saw further innovation with the rise of social media platforms like LinkedIn and the widespread adoption of Applicant Tracking Systems (ATS). These systems automated many administrative tasks, such as resume screening and interview scheduling, facilitating strategic talent acquisition and improved communication channels between recruiters and candidates. (Zuker, 2023)

The 2010s were characterized by the integration of more sophisticated technologies, such as artificial intelligence (AI) and machine learning, which automated repetitive tasks and mitigated unconscious biases in hiring decisions. Recruitment practices have become increasingly data-driven, with analytics being used to optimize hiring efficiency and effectiveness. Video interviews also became more commonplace, enabling companies to engage with candidates globally and streamline selection. (Zuker, 2023)

The COVID-19 pandemic accelerated these digital trends, making remote interviews and virtual onboarding the norm. Companies increasingly relied on digital tools to connect with candidates and maintain engagement throughout the hiring process. Data-driven insights reveal significant changes in applicant behaviour, particularly the increasing use of mobile devices for job searches. Many job seekers now use smartphones to explore and apply for jobs, highlighting the necessity for mobile-friendly application processes. Moreover, social media has become a vital recruitment tool, emphasizing the importance of recruiters maintaining an active and engaging online presence. (Solutions, 2024)

AI's role in recruitment has also grown, enhancing job matching and improving both the efficiency of the recruitment process and the satisfaction of HR personnel. By automating operational tasks, AI allows recruiters to focus more on engaging with suitable candidates, increasing the quality of hires and candidate satisfaction.

The evolution of recruitment practices reflects a broader shift from manual, localized efforts to a highly efficient, global, and technologically driven process. This transformation has expanded the scope and speed of recruitment and introduced new tools and platforms that continue to shape how companies and candidates connect in an increasingly digital world.

1.2 Problem Statement

In today's rapidly evolving job market, the recruitment process faces significant challenges that impact employers and job seekers, leading to inefficiencies, increased costs, and missed opportunities for optimal employment matches. A primary concern is handling the high volume of job applications received for each opening. Research indicates that a single job posting can attract an average of 250 applicants, but only a fraction—approximately 4 to 6—are invited for an interview (G. Team, 2018). This underscores the competitive nature of job markets and highlights the daunting task that HR departments face in managing such volumes effectively.

One of the critical issues exacerbating these challenges is the prevalence of unclear job descriptions. Without precise specifications, applicants find it challenging to ascertain whether they meet the qualifications or would fit the organisational culture, leading to a mismatch between job offerings and job seeker capabilities.(UpJourney, 2023) This ambiguity results in a glut of applications, many from candidates unsuitable for the role, thereby straining the screening process and extending the time required to identify suitable candidates. (Reporter, 2020) Furthermore, the Society of Human Resource Management reports an average cost per hire of over \$4,000, which varies significantly across industries but universally adds financial strain to companies (Navarra, 2023)

The length of the recruitment process itself poses additional complications. On average, the hiring process spans three to six weeks (Lora, 2023). During this time, nearly 60% of candidates lose interest and abandon their applications if the process is perceived as overly lengthy or complex.(Hiredna, 2023) This attrition is detrimental because it can cause companies to miss out on top talent and reflects poorly on the company's brand, potentially deterring future applicants.

Moreover, job seekers frequently express frustration with the recruitment process. They face constant rejections, which is often disheartening and demotivating. The challenge is compounded by a digital environment overwhelmed with opportunities that do not necessarily align with the seekers' skills or career aspirations. For those actively seeking new positions, the necessity to sift through numerous listings to find relevant posts can be time-consuming and inefficient, often resulting in missed opportunities for both parties.(Global, 2022)

The need for innovation in this space is clear: a sophisticated, AI-driven job-matching system could address these inefficiencies head-on. By leveraging advanced algorithms and natural language processing (NLP), such a system would streamline the application and screening processes and ensure a more accurate match between job descriptions and applicant profiles. This would reduce the volume of mismatched applications, decrease hiring costs, and shorten the hiring cycle. Furthermore, automating initial screenings could minimise the influence of unconscious biases, promoting a more diverse and inclusive workplace.

In summary, the recruitment process as it currently stands is fraught with inefficiencies that necessitate a technological intervention. An advanced, automated system could dramatically

transform how companies and job seekers connect, making the process quicker, less costly, and more equitable. This project proposes to develop such a system, addressing the core inefficiencies identified in the recruitment process, thereby enhancing the experience and outcomes for all stakeholders involved.

1.3 Objective

A common issue in recruitment is unclear job descriptions, which can lead job seekers to apply for positions without knowing if they meet the necessary qualifications or would be a good fit for the company culture. This often results in a mismatch between job seekers' capabilities and job requirements. **Dynamic ranking mechanisms can be employed based on advanced matching algorithms** like cosine similarity, TF-IDF, and word embeddings (Word2Vec, GloVe). By vectorizing job descriptions and resumes, these algorithms can assess the similarity and relevance between a candidate's profile and the job's requirements. Based on this calculated compatibility, the system can rank job listings for each job seeker. Such a ranking makes it clear to the job seeker where they stand regarding meeting the qualifications and helps them prioritize their applications accordingly. This targeted approach reduces confusion and increases efficiency in the application process.

the issue of job seeker frustration with constant rejections due to mismatches between their qualifications and job requirements can be alleviated using **advanced matching algorithms**. Technologies like cosine similarity, TF-IDF, and word embeddings (Word2Vec, GloVe) can enhance the precision of matching candidates to job descriptions. These algorithms analyze the textual and semantic similarities between resumes and job postings, ensuring higher relevance in matches. This targeted matching reduces the incidence of job seekers being rejected. It improves their experience by connecting them with suitable job opportunities that align with their skills and career aspirations.

A significant problem in recruitment is the extensive time required to process a high volume of applications, many of which come from candidates needing more clarification on their qualifications for the role. This results in inefficiencies and extended hiring times. To address this, the **dynamic ranking mechanism**, which uses advanced matching algorithms, can be particularly effective. By ranking job listings based on the compatibility scores calculated from resume and job description similarities, job seekers can immediately identify which positions they are most suited for. This helps candidates focus their applications on roles where they have a higher chance of success and reduces the number of unsuitable applications companies receive. Consequently, HR departments can expedite the screening process, significantly shortening the recruitment timeline.

A dynamic ranking mechanism in the job application interface streamlines the process for both parties. For employers, it means receiving applications that are more likely to be from qualified and genuinely interested candidates, thus enhancing the quality of the applicant pool. For job seekers, it means spending less time on broad searches and unnecessary applications, focusing instead on opportunities where their chances are best, based on aligning their skills and qualifications with the job requirements.

The aim of the project is to give some power to the candidate/job seeker, where most resume ranking systems have needed to improve in the past; the resume system only

prioritises the Employer. When the job seeker uploads their resume on the platform, it will rank the job listing to the user before the user sends their resume to a specific job that interests them. In return, the job seeker can save time on job hunting as the time needed to scroll through all the job listings is lessened. At the same time, the Employer would be more likely to receive a better quality resume as in the past; job seekers might apply for a job that is not well-fit for the job listing due to the unknown of whether they are a good fit. Now, with the recommendation system provided to the job seeker, they are more certain about which job is more likely to align with their skills and experience.

1.4 Scope of Study

The scope of this study encompasses developing and implementing a job-matching platform tailored specifically for the technology sector. This project uses advanced data processing and matching algorithms to enhance the recruitment process. Below is a detailed description of what is included in the report and the limitations regarding the technologies that are not incorporated.

1.4.1 Included in the Study:

Text Similarity Measures:

- **Cosine Similarity:** Assesses the textual similarity between job descriptions and resumes by measuring the cosine of the angle between two vectors in a multi-dimensional space.
- **TF-IDF (Term Frequency-Inverse Document Frequency)** weighs the importance of words in documents, enhancing the accuracy of matching job descriptions and resumes.

Specific Technologies and Tools:

- **Natural Language Processing (NLP) Libraries (e.g., NLTK, spaCy):** These are used for text normalization processes, including tokenization, removing stopwords, and other preprocessing steps.
- **Natural Language Processing (NLP) Libraries For Named Entity Recognition (NER) (NLTK, spaCy, and Stanford NER):** These are utilized to extract critical information from resumes for advanced NLP tasks, including NER. These libraries identify and classify critical entities like names, job titles, and organizations within text data.

Supervised Machine learning Algorithm:

The project incorporates several supervised machine learning algorithms to categorize job listings into specific job types, **such as engineering, accounting, and information technology**. These algorithms include:

- **Support Vector Machine (SVM):** Chosen for its effectiveness in high-dimensional spaces, SVM is utilized to accurately classify job types by creating a clear margin that separates different categories, making it ideal for the diverse features found in job descriptions.
- **Logistic regression:** This algorithm is implemented for its capability in probabilistic classification, which is pivotal for determining the likelihood of a job listing belonging to

each job type category. It is beneficial for providing insights into the confidence level of predictions.

- **Random Forest:** selected for its robustness and accuracy, Random Forest uses multiple decision trees to enhance prediction accuracy and prevent overfitting. This algorithm handles the variability and complexity inherent in job data, offering reliable categorization across various job types.

Data Storage:

The project utilizes a secure, cloud-based MongoDB database system to store and manage critical data components, which are fundamental to the functionality of our job-matching platform. The database is specifically designed to:

- **Securely store user credentials**, including **Employer usernames** and **passwords**. This facilitates robust user authentication and access management, ensuring secure and controlled access to the platform.
- **House comprehensive details about job listings**, such as **job titles**, **descriptions**, **addresses**, and **benefits**. This setup enables efficient job search functionalities and effective job posting management, contributing to an optimized user experience and operational efficiency.

1.4.2 Excluded from the Study:

Broad Industry Application:

This project's scope is intentionally focused on three distinct sectors: Technology, Engineering, and Accounting. These industries were selected due to their reliance on specific skill sets and terminologies, which lend themselves well to analysis through text similarity measures like TF-IDF and Cosine Similarity. Such sectors are characterized by job descriptions rich in keywords and technical language, making them ideal for extracting meaningful features using these methods. This keyword-oriented approach enhances the accuracy of job matching by precisely aligning candidates' skills and qualifications with the job requirements. While other industries also benefit from sophisticated job-matching techniques, the characteristics and recruitment needs of the Technology, Engineering, and Accounting sectors make them particularly suitable for demonstrating the efficacy of the implemented algorithms.

Non-Persistent Resume Storage Policy:

The project adopts a non-persistent resume storage policy, requiring job seekers to upload their resumes each time they engage with the platform. This strategy enhances the system's flexibility, making it suitable for various settings from large job-seeking websites to university internship portals, and allows for easy scalability without significant architectural changes. It also cuts costs

by reducing data storage needs, which is particularly advantageous for smaller entities or educational institutions with tight budgets. Additionally, by ensuring resumes are uploaded afresh, the system consistently works with the most up-to-date and relevant information, increasing the accuracy of job matches and eliminating the need for users to update stored resumes, thus maintaining alignment with their current professional status and objectives. This approach optimally supports the practical and financial considerations of its intended user base, ensuring an accurate and efficient job-matching process.

1.5 Chapter Summary and evaluation

First chapter outlines developing an AI-driven job-matching system to enhance recruitment by addressing inefficiencies and challenges in the current process. It traces the evolution of recruitment from traditional methods to the integration of AI, highlighting the benefits of technological advancements. Current problems include high application volumes, unclear job descriptions, lengthy hiring processes. The project aims to improve matching accuracy using advanced algorithms like cosine similarity and TF-IDF, reducing mismatches and promoting diversity. The scope focuses on the technology sector, employing NLP tools for resume and job description analysis. This approach seeks to streamline recruitment, enhance candidate experience, and increase hiring efficiency.

The second chapter reviews technologies and methodologies for developing job-matching platforms in the technology, Accounting and engineering sectors, focusing on text similarity measures techniques like TF-IDF and Cosine Similarity and Named Entity Recognition (NER). Cosine similarity assesses document similarity by measuring the angle between vectors, while TF-IDF weighs term importance across documents. NER uses AI to identify and categorize key information in text. The chapter also examines existing solutions, highlighting projects by Tijare et al., Roy et al., and Jagwani et al., which utilized these techniques to enhance recruitment processes. These projects demonstrated significant successes but also faced limitations, such as handling varied CV formats and improving accuracy for certain extractions.

The third chapter outlines the workflow, software, algorithms, and technologies used in developing the job-matching platform. It covers data preprocessing steps, including stopword removal, case normalization, Punctuation removal, whitespace normalization and lemmatization, to ensure clean data. Tokenization splits text into manageable units, while TF-IDF vectorization converts text into numerical formats for machine learning processing. Named Entity Recognition (NER) identifies and categorizes key information from resumes. Cosine similarity measures the alignment between resumes and job descriptions, facilitating accurate job matching. Data storage uses MongoDB for efficient handling and searching of large text volumes. Job seekers' resumes and employers' job descriptions are stored in cloud storage systems, ensuring data retention and quick processing for real-time comparisons. This

methodology aims to streamline recruitment, providing better job matches and improving the experience for both candidates and employers.

Chapter Four delineates the comprehensive system workflow, beginning with job seekers uploading their resumes in PDF format, processed using PDFPlumber to accurately extract textual data. To ensure the integrity and cleanliness of the data, both resumes and job descriptions undergo several preprocessing steps. This includes transforming all text to a uniform case using case normalization, removing common stopwords utilizing NLTK to focus on more meaningful words, and applying lemmatization with WordNetLemmatizer to reduce words to their base or dictionary form. Additionally, Named Entity Recognition (NER) via spaCy is employed to detect and classify key information from the text such as names and technical skills. The system's interactive user interface caters to both job seekers and employers. It allows job seekers not only to upload and manage their resumes but also to utilize advanced filtering options to navigate through job listings effectively. They can select specific job criteria and use the "Find Best Match" feature, which leverages cosine similarity to rank and display jobs that best match their profile based on the preprocessed TF-IDF data. For employers, the system provides secure registration and login functionalities, enabling them to submit and manage job postings easily. Once logged in, employers can create new job listings, read and update existing ones, or delete them, ensuring they have full control over the content they provide. This dual-interface approach ensures that both job seekers and employers can interact seamlessly with the system, enhancing the recruitment process and making it more efficient and user-friendly.

Chapter Five offers an in-depth examination of the job-matching system's implementation, focusing on the user interfaces for job seekers and employers, and the visualization tools that enhance the platform's interactivity. This chapter meticulously outlines the construction of each interface, including specific functionalities such as job listing filters and employer job management options. It details the system's backend operations, including the database configuration and security protocols that ensure data integrity and protection. Additionally, it provides a thorough overview of the testing strategies employed to verify the system's performance and reliability across various scenarios, ensuring that each component functions correctly within the system's architecture. This comprehensive approach underscores the system's readiness and efficiency in streamlining the job-matching process.

The seventh chapter of this thesis encapsulates a comprehensive evaluation and reflection on the AI-driven job-matching system developed to enhance the recruitment process. By leveraging advanced natural language processing techniques such as TF-IDF and cosine similarity, the system efficiently matches job descriptions with applicant profiles, significantly reducing hiring times and costs and

promoting a more inclusive hiring landscape. Despite successfully achieving its primary objectives, the project encountered challenges like data inconsistency and integrating multiple technologies, which were mitigated through rigorous testing and modular development approaches. Nonetheless, limitations exist, notably the system's initial setup allowing unrestricted IP access, which poses security concerns for production environments. This highlights future enhancement opportunities, such as implementing IP whitelisting and expanding user testing. Moreover, the system's market potential extends to universities and job-hunting platforms, offering substantial benefits in streamlining the job-seeking process. This chapter provides a critical overview of the project's achievements, contributions, and valuable lessons learned, emphasizing the importance of data quality and security in AI-based systems.

Chapter 2

Literature Review

2 Literature Review

This chapter comprehensively reviews existing literature on technologies and methodologies relevant to developing job-matching platforms in the technology sector. It includes discussions on unsupervised machine learning techniques like **TF-IDF** and **Cosine Similarity**, **Named Entity Recognition (NER)**. Additionally, it examines similar platforms and solutions developed by other researchers and companies, highlighting their approaches, successes, and limitations to contextualize the current study's objectives and contributions.

2.1 Technology involved

2.1.1 Cosine similarity

Cosine similarity is a method used to assess how similar two vectors are by focusing on the angle between them rather than their length or size. This metric is beneficial in comparing documents irrespective of their length, as it measures the cosine of the angle formed by two vectors in a multi-dimensional space, where each vector represents the word frequency in a document.(Fatih, n.d.) Unlike methods that count common words, which can misleadingly suggest similarity when documents are lengthy and share many words, cosine similarity evaluates the directional alignment of the vectors. This means that even if two documents differ significantly in size or the frequency of specific words, they can still be considered similar if they share a similar orientation or direction in this multi-dimensional space. Thus, cosine similarity is beneficial for capturing the essence of document similarity in cases where traditional distance measures like Euclidean distance might fail due to differences in document length.(Prabhakaran, 2022)

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} :$$

θ is the angle between the vectors,

Figure 1 The formula for cosine similarity

where A and B are vectors, $A \cdot B$ is the dot product of the vectors, and $\|A\|$ and $\|B\|$ are the magnitudes (or Euclidean norms) of the vectors. When two vectors are aligned in the same direction, their angle is zero degrees, resulting in a cosine similarity of 1. Vectors perpendicular to each other have a 90-degree angle in between, giving a cosine similarity of 0. Conversely, if

vectors point in precisely opposite directions, the angle between them is 180 degrees, and their cosine similarity is -1.(Fatih, n.d.)

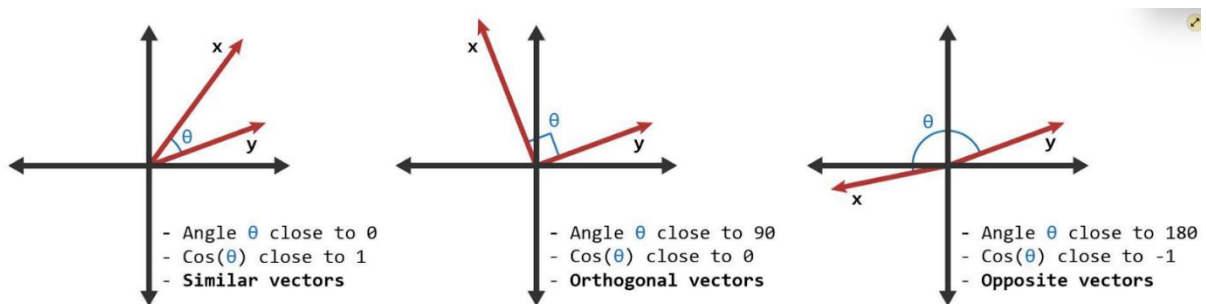


Figure 2 Graphic showing vector with similarities close to 1, close to 0 and close to -1

Cosine similarity finds wide application in Natural Language Processing, aiding in gauging the semantic similarity between words. Its utility extends to sentiment analysis, text classification, and summarization tasks. Similarly, in recommendation systems, it proves invaluable for determining if two users share preferences for similar content. By leveraging cosine similarity, these systems can suggest comparable items to users based on their preferences.(Antoniadis & Antoniadis, 2023)

There are multiple libraries that support built-in cosine similarity functions to compute similarity between documents, such as Gensim (Fares, 2022), Spacy (Foong, 2021), NLTK (Natural Language Toolkit) (GeeksforGeeks, 2023) and Scikit-Learn(Katarina, 2023).

2.1.2 Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF (Term Frequency-Inverse Document Frequency) is a powerful tool appreciated for its simplicity, computational efficiency, and ease of implementation. It effectively differentiates word importance by weighting terms based on frequency and scarcity across documents, offering more depth than the essential word count metrics of the Bag of Words model. However, it is important to note that TF-IDF does not capture words' semantic meaning or context. It also neglects the significance of word order, which can alter meanings in phrases like "pay the bill" or "not pay the bill." These limitations can be mitigated by using advanced NLP techniques like treating phrases as single units using named entity recognition tools. (Anirudha Simha, 2021)

The model also needs help with memory efficiency due to the curse of dimensionality, as the size of TF-IDF vectors grows with the vocabulary size, which can become cumbersome in scenarios like clustering with many documents. Turning to more advanced NLP techniques like BERT or Word2Vec might be beneficial in such cases.(Arya, n.d.-b)

Term Frequency (TF)

Term Frequency measures the frequency of a word in a specific document. It is calculated as:

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

Figure 3 Formula for Term Frequency (TF)

Term Frequency (TF) is the count of times a specific term appears in a document divided by the total number of terms. This metric assesses the relative importance of a word within a single document, accounting for variations in document length. For example, a word that appears three times in a 10-word document is considered more significant than if it appears the same number of times in a 100-word document. By normalizing word counts in this way, TF reflects the importance of a word relative to the document's size, emphasizing its significance more in shorter documents where it appears frequently.(Chouksey, 2021)

Inverse Document Frequency (IDF)

Inverse Document Frequency measures the rarity of a term across documents in the corpus. It is calculated as:

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

Figure 4 Inverse Document Frequency (IDF) Formula

Inverse Document Frequency (IDF) gauges how rare or common a word is across a collection of documents, known as a corpus. IDF assigns higher weights to words that appear in fewer documents, indicating their unique informational value. Conversely, common words like "a" and "the," which appear in nearly all documents, receive lower IDF values. The formula for IDF is the logarithm of the total number of documents in the corpus divided by the number of documents that contain the specific term. This logarithmic scaling ensures that terms only found in a few documents are emphasized as more significant, reflecting their rarity. Essentially, IDF measures the exclusivity of a word's occurrence across the corpus, highlighting words that are not widespread and thus may carry more unique meanings in particular contexts.(Chouksey, 2021)

TF-IDF Calculation

The **TF-IDF** value is calculated by multiplying TF and IDF for each term:

$$TF-IDF = TF * IDF$$

Figure 5 Formula for TF-IDF

Combining these two metrics means that a word gets a high TF-IDF score if it appears frequently in a particular document but not commonly across multiple documents. This balance allows TF-IDF to highlight words that are potentially more relevant and specific to the content of one document relative to others in the corpus. Essentially, TF-IDF helps identify and weigh terms based on their uniqueness and relevance to a single document within the larger document set.(Chen, 2022)

TF-IDF is supported by the Machine learning library, which has a built-in TfidfVectorizer for computing Term Frequency-Inverse Document Frequency (TF-IDF) scores.(Chaudhary, 2021)

2.1.3 Named Entity Recognition (NER)

Named Entity Recognition (NER) is an AI technique used in natural language processing (NLP) that identifies and categorizes key information in text into predefined groups, such as names of people, organizations, locations, and times. By identifying these entities, NER provides a structured understanding of the text's content, making it easier to analyze large volumes of data. (Marshall, 2020)

Named Entity Recognition (NER) systems can be categorized into four main types based on their underlying methodologies. **Dictionary-based** systems use predefined lists of words from specific fields or domains to identify entities within text. **Rule-based** systems depend on specific rules that dictate entity recognition based on word patterns and contextual clues, such as recognizing honorific titles preceding names, which can be effective but often lack flexibility and scalability. **Machine learning-based** systems utilize statistical models trained on annotated documents to identify and categorize entity names effectively. These documents contain marked entities that teach the model to recognize similar patterns in new texts. Employ algorithms that can learn from data, making them more adaptable to new, unseen text but requiring substantial amounts of labelled training data. Lastly, **hybrid systems** combine elements from the above approaches, leveraging rule-based strategies and machine learning to enhance accuracy and adaptability in identifying entities across diverse texts. (Schulze, 2024)

Advanced **machine learning models** like **conditional random fields (CRFs)**, deep learning models such as **recurrent neural networks (RNNs)**, and transformers like **BERT** are increasingly popular in NER due to their ability to handle the complexity and variability of language. These models are particularly adept at considering the context of words, which is essential for accurately recognizing and classifying entities. (What Is Named Entity Recognition? | IBM, n.d.) NER is utilized across various industries for different applications. It helps

streamline processes by extracting meaningful information from texts, such as summarizing resumes in HR, enhancing customer support systems by categorizing complaints or improving information retrieval in academic research. In healthcare, for instance, NER can extract patient information from clinical reports, aiding in more efficient data management. (Marshall, 2021) To implement NER, one can utilize popular NLP libraries such as **NLTK**, **SpaCy**, and the **Stanford NER**, which offer pre-trained models and tools to customize NER systems according to specific needs. These tools provide a good starting point for organizations looking to integrate NER into their operations, allowing for the rapid processing and analysis of text to extract meaningful and actionable insights. (*What Is Named Entity Recognition?* | IBM, n.d.)

The process of Named Entity Recognition (NER) begins with collecting an annotated dataset, where text examples are labelled with named entities to indicate their types, either manually or through automated methods. This is followed by data preprocessing to clean and format the text, removing unnecessary characters and splitting it into sentences or tokens. Next, relevant features such as part-of-speech tags, word embeddings, and contextual information are extracted based on the needs of the specific NER model. The model is then trained on this prepared dataset to learn patterns and relationships between words and their corresponding entity labels. After training, the model's performance is evaluated using metrics like precision, recall, and F1 score to assess its ability to identify and classify entities accurately. Depending on the results, the model may undergo fine-tuning to enhance its performance, involving adjustments to hyperparameters or the training approach. Once refined, the model is ready for inference on new, unseen text, applying all preprocessing steps and feature extractions to predict entity labels. Finally, the output might require post-processing to refine results and link identified entities to external knowledge bases or databases for further contextual enrichment. For accurate categorization, extensive and relevant training data are crucial, as they help the model learn the characteristics of different entities. (*What Is Named Entity Recognition?* | IBM, n.d.)

2.1.4 Support Vector Machine (SVM)

Support Vector Machines (SVMs) are a robust set of supervised machine learning algorithms predominantly used for classification, though they are also effective for regression tasks. In classification tasks, SVMs aim to identify the optimal hyperplane that maximizes the margin between different data classes in an N-dimensional space. This hyperplane acts as a decision boundary, adept at classifying new data points into predefined categories based on their attributes. Classification with SVM involves categorizing data into discrete labels, making it suitable for tasks like email spam detection or determining whether a tumor is malignant or benign.

On the other hand, in regression tasks, SVMs are used to predict a continuous output variable from one or more input features. Instead of finding a decision boundary that separates the data into classes, SVM regression aims to fit the best line or curve that captures the trends in the data as closely as possible while also considering the margins of error in the predictions. This makes SVM regression useful for tasks such as predicting house prices or stock market trends, where the output is a continuous value rather than a discrete category. (Achsan, 2021)

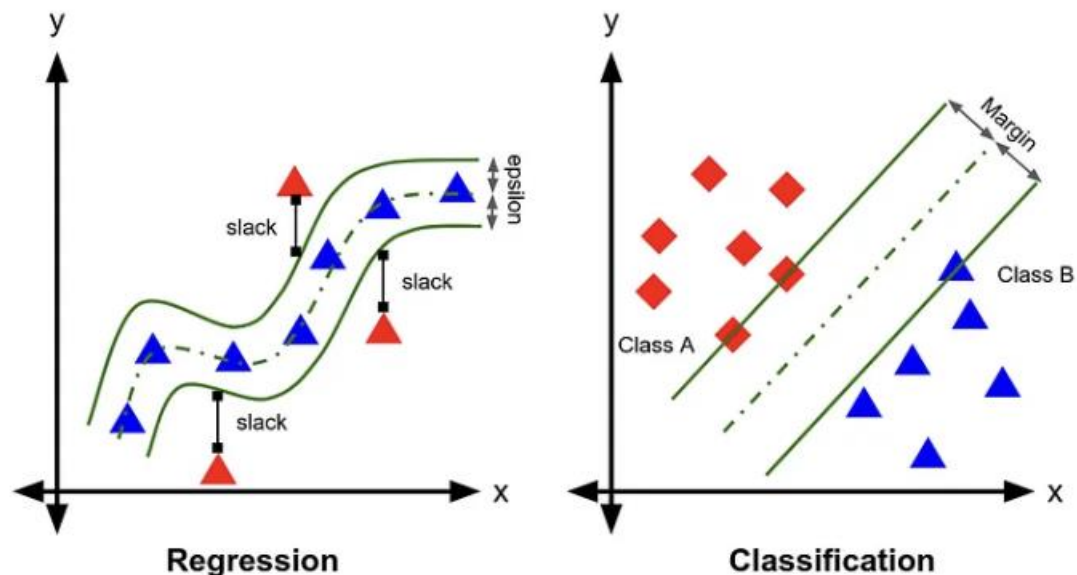


Figure 6 Classification VS regression Of SVM

How it work

Linear SVM is utilized when the data points are linearly separable, meaning they can be divided by a single straight line (in two dimensions) or a hyperplane (in higher dimensions).

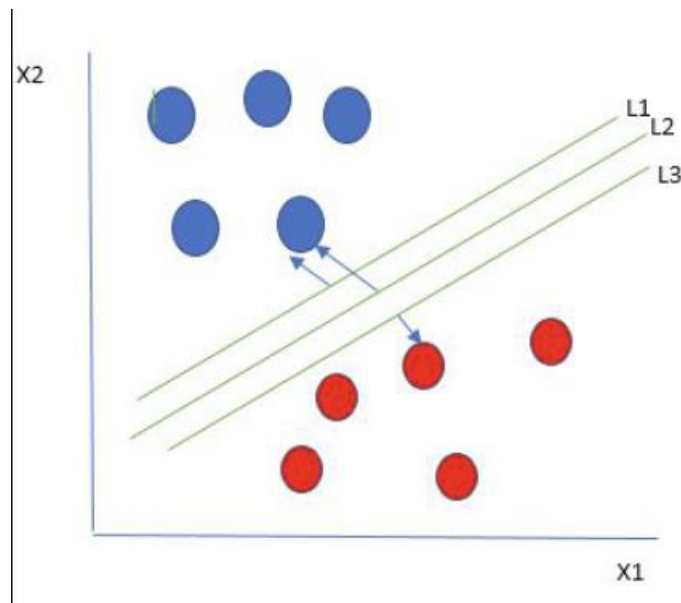


Figure 7 SVM Hard Margin

A sensible choice for the optimal hyperplane is one that achieves the greatest separation, or margin, between the two classes. This is accomplished by selecting a hyperplane that maximizes the distance to the nearest data point from each class. If such a hyperplane exists, it is referred to as the maximum-margin hyperplane, or hard margin. From the referenced figure, the ideal choice would be L2. The hard margin hyperplane is defined as one that distinctly separates the data points of different classes without any errors or misclassifications. (GeeksforGeeks, 2024)

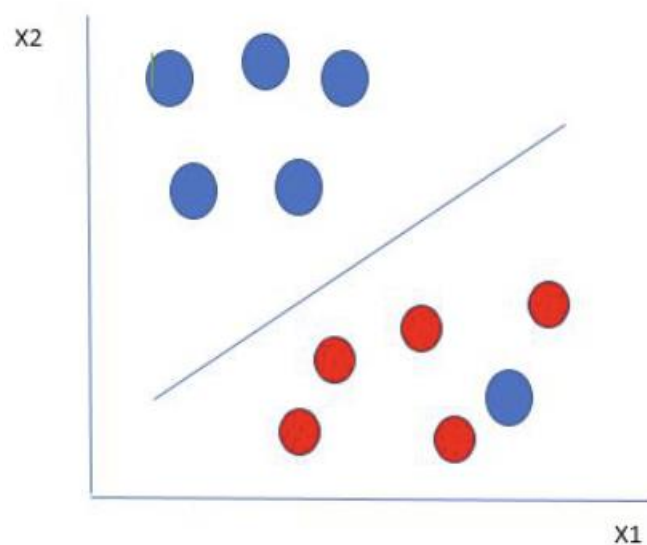


Figure 8 SVM Soft Margin

When faced with data points like a blue ball within the boundary of red ones, SVM identifies it as an outlier and focuses on finding the optimal hyperplane that maximizes the separation margin, showcasing its robustness to outliers. In such scenarios, SVM employs a soft margin approach that allows for flexibility by introducing penalties for each data point that crosses the margin. These penalties are quantified using hinge loss, a loss function that increases with the distance of misclassified points from the decision boundary, thereby penalizing incorrect classifications and margin violations. Slack variables are also introduced, which allow some data points to violate the margin requirements, thus permitting minor misclassifications to achieve a balance between maximizing the margin and minimizing these violations. The combination of hinge loss with a regularization term in SVM's objective function further aids in avoiding overfitting, ensuring the model remains general and effective on new data.(GeeksforGeeks, 2024)

Non-linear SVM comes into play when the data is not linearly separable, exhibiting a more complex relationship that cannot be captured by a straight line. To address this, Non-linear SVM uses kernel functions to map the original non-linear observations into a higher-dimensional space where a linear separation is possible.(GeeksforGeeks, 2024)

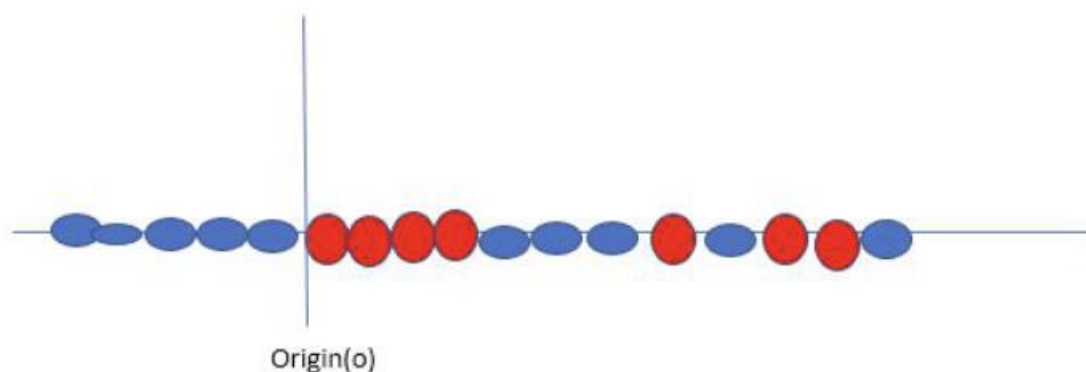


Figure 9 Non-linear SVM

When dealing with data that isn't linearly separable, such as when groups of blue and red balls cannot be divided by a straight line.

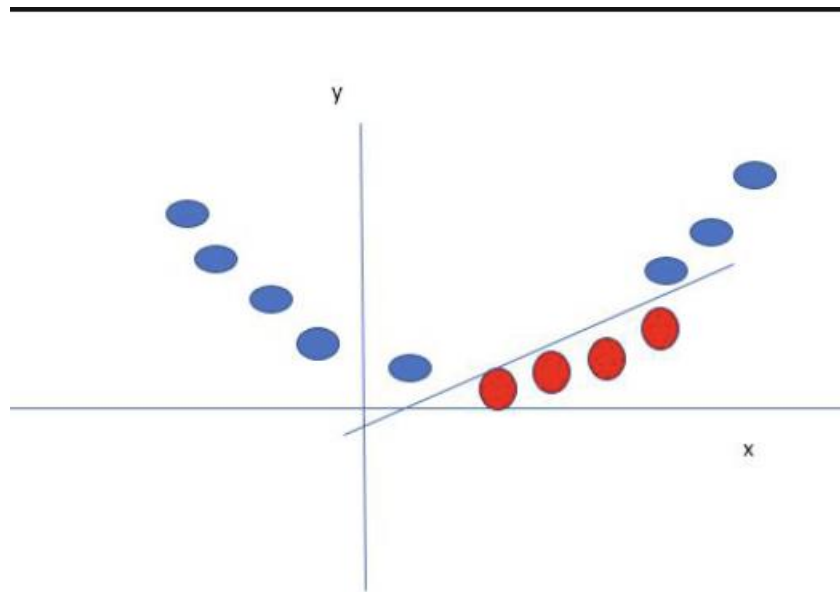


Figure 10 Non-linear SVM using Kernal Function

Support Vector Machines (SVM) utilize a technique involving kernel functions to address this challenge. A kernel is a mathematical function employed by SVMs to transform the original input data points into a higher-dimensional feature space. This transformation facilitates the identification of a hyperplane that can effectively separate the data, even if the separation is not possible in the original input space. The kernel function essentially redefines the dataset in a new dimensional space where the points are linearly separable. Commonly used kernels include the linear, polynomial, radial basis function (RBF), and sigmoid kernels. Each kernel varies in how it manipulates the space and complexity of the separation boundary it can produce, making them suited for different types of nonlinear data. This method allows SVMs to extend their robust classification capabilities beyond linear relationships without a substantial increase in computational complexity. (GeeksforGeeks, 2024)

Mathematical intuition of SVM:

The equation for linear hyperplane

$$w^T x + b = 0$$

Figure 11 The equation for linear hyperplane

The equation for the linear hyperplane is expressed using the vector $\mathbf{W}$, which represents the normal to the hyperplane, meaning it is perpendicular to the surface of the hyperplane. The parameter $\mathbf{b}$ in the equation indicates the offset or the distance from the origin to the hyperplane along the normal vector $\mathbf{W}$. This formulation helps define the hyperplane's position and orientation in the feature space.

The equation for Decision Boundary

$$d_i = \frac{w^T x_i + b}{||w||}$$

Figure 12 **The equation for linear hyperplane**

where $||w||$ represents the Euclidean norm of the weight vector w . Euclidean norm of the normal vector W

2.1.5 Logistic regression

Logistic regression is a predictive modeling algorithm used extensively in statistics and machine learning for binary classification tasks. It estimates the probability of a binary response based on one or more predictor variables by utilizing a logistic function. This logistic function, also known as the sigmoid function, maps any real-valued number into the (0, 1) range, making it particularly suitable for models where the output is a probability. One of the main advantages of logistic regression is its high interpretability; coefficients in a logistic regression model represent the change in the log odds of the dependent variable for a one-unit change in the independent variable, holding other variables constant. Additionally, logistic regression is computationally efficient and straightforward to implement, which makes it a popular choice for many binary classification problems. It also outputs probabilities, a feature that can provide more nuanced insights into results than simply the final classification. (Kfollis, 2023)

However, logistic regression does come with its limitations. It assumes a linear relationship between the logit of the outcome and each predictor variable, which might not always hold true, thus restricting its effectiveness with non-linear datasets. It may not perform well with complex relationships without transformation of variables or adding interaction terms. Logistic regression models can also be sensitive to imbalanced data sets, leading to biased predictions favoring the majority class. There are different variations of logistic regression to handle specific types of data: Binary Logistic Regression, used when the dependent variable is binary; Multinomial Logistic Regression, which is applicable when the dependent variable is categorical with more than two categories; and Ordinal Logistic Regression, suitable for ordinal dependent variables. These variations enhance the applicability of logistic regression across a broader spectrum of classification problems, making it a versatile tool in the statistical learning arsenal. (Yasar et al., 2024)

2.1.6 Random Forest

Random Forest is a versatile and powerful machine learning algorithm that operates by constructing a multitude of decision trees at training time and outputting the class that is the mode of the classes

(classification) or mean prediction (regression) of the individual trees. It builds multiple decision trees and merges them together to get a more accurate and stable prediction, which significantly enhances the accuracy of the model while preventing overfitting, a common problem in decision trees. Random Forest can handle large datasets with higher dimensionality and can estimate which variables are important in the classification. It is also robust to outliers and can handle unbalanced data effectively. However, it is computationally expensive and less interpretable compared to other algorithms, since the ensemble nature of the model makes it hard to understand which decision tree contributed to the final decision. (What Is Random Forest? | IBM, n.d.-b)

Random Forest comes in two types: Random Forest Classifier for categorical targets and Random Forest Regressor for continuous targets. These variations allow it to be applied flexibly across different types of data analysis scenarios. Despite its power, Random Forest can be slow to create predictions once trained, especially with large numbers of trees. It also does not perform well on very sparse data like text data, where models like Naive Bayes might be more appropriate. Nevertheless, its ability to provide feature importances which can be insightful for understanding the significant predictors in the dataset, and its robustness against overfitting when dealing with datasets featuring numerous variables, make it a valuable tool for many predictive tasks. (Simplilearn, 2023)

2.2 Similar Project

This research paper delves into the development of an advanced job recommendation system designed to optimize the recruitment process by automating the screening of resumes, a necessity in India's vast job market where manual review processes often lead to inefficiencies and high hiring costs. Resumes, varying widely in format and content, pose significant challenges for recruitment teams tasked with manually matching candidates to job descriptions. The proposed system utilizes machine learning and natural language processing to classify resumes according to job descriptions and calculates a similarity percentage to determine candidate suitability. Only candidates whose resumes match the job description by at least 50% are considered further, significantly reducing the likelihood of overlooking qualified applicants. This approach not only speeds up the recruitment process but also minimizes the resources expended in finding the right candidate, thereby enhancing the effectiveness of a company's recruitment efforts. By integrating classification techniques and domain knowledge, this system aims to streamline the hiring workflow, making it more efficient and less prone to errors, and ultimately adding value to the company's recruitment initiatives. (Anushka & Senthilkumar, 2022)

In developing automated resume screening systems, researchers have employed sophisticated machine learning (ML) and natural language processing (NLP) techniques to enhance the recruitment process. One such project utilized the BERT model for name extraction, achieving a high accuracy rate of 98% and an F1-score of 0.70 for Indian names. This model was trained with a dataset of over 30,000 names and involved padding the POS-tagging sequence to fit BERT's input requirements. Additionally, the system included a Basic Details Extractor, using regular expressions for phone numbers and email addresses, which attained 100% accuracy, and for profile statements and experience, which reached 91% accuracy. An Education Details Extractor, leveraging noun chunking and pattern matching, identified educational institutions and CGPAs with 94% accuracy. Text extraction from resumes in PDF and DOCX formats was performed using pdfminer, followed by text cleaning and tokenization. Despite these successes, the system faces limitations such as enhancing experience extraction accuracy and expanding to process multiple languages. Future improvements aim to address these challenges, enhancing the model's robustness and applicability (Tijare et al., 2023) .

The study by Roy et al. (2020) takes a novel approach to the critical issue of efficiently identifying the right talent by automating resume screening. This unique method involves preprocessing resumes through cleaning, tokenizing, stop words removal, stemming, and lemmatization, followed by feature extraction using Tf-Idf. Supervised learning models, including a Linear Support Vector Classifier, were employed, achieving an accuracy of 78.53%. Additionally, recommendation models using Cosine Similarity and k-nearest Neighbors were developed to match resumes to job descriptions. The project leveraged technologies such as Python, NLTK, scikit-learn, TensorFlow, and the "gensim" library. While the study demonstrated significant success in automating resume matching, it highlighted limitations, including handling varied CV formats and potential information loss during summarization. Future improvements were suggested, such as integrating deep learning models and using industry-specific data to enhance the model's performance (Roy et al., 2020) .

The proposed system by Jagwani et al. (2023) automates resume evaluation by combining Latent Dirichlet Allocation (LDA) and Named Entity Recognition (NER) for improved accuracy and efficiency. It collects resumes in PDF/DOCX format, extracts text using SpaCy, and preprocesses it for analysis. LDA identifies vital topics and keywords within resumes, while SpaCy's NER extracts essential entities like names, contact information, education, work experience, and skills. These extracted elements are used to parse and rate resumes, resulting in a final score out of 10. This method achieved 82% and 77% accuracy for skill identification, surpassing traditional keyword-based approaches. Limitations include needing a larger, more diverse dataset and

potential improvements using advanced models like BERT and incorporating user feedback for continuous refinement.

Chapter 3

Methodology and Requirements Analysis

3 Methodology and Requirements Analysis

This section gives an overview of the system's workflow, how it works, and all the potential software, algorithms, and technology being used in the project. It also explains the potential outcomes and requirements needed in each stage of the project. Below is a simplified workflow of the system.

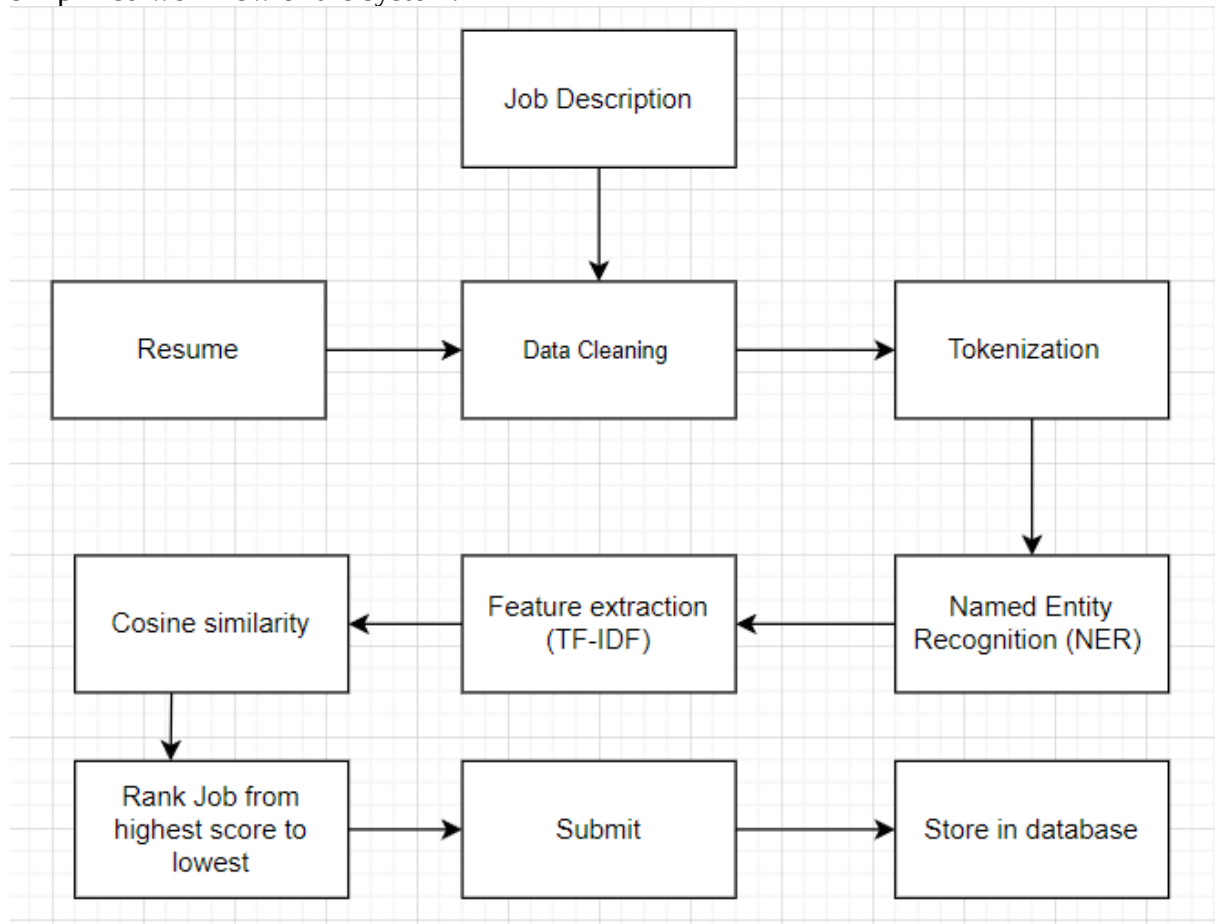


Figure 13 simplified workflow of the system.

3.1 Preprocessing

3.1.1 Data Cleaning

Stopword removal, **case normalization**, **Lemmatisation**, and **stemming** are applied to the resume uploaded by the job seeker and job description uploaded by the Employer to ensure the noise of the data is removed; all subsequent analysis can be performed on clean and relevant data. Below is how each technology is applied, why it is applied, and which library or function supports it.

3.1.1.1 Stop word removal

In natural language processing (NLP), stopwords are frequently occurring words such as "the," "is," and "and," and generally carry minimal semantic meaning. They are considered 'noise' in text analysis and are often removed during preprocessing to enhance the efficiency and accuracy of various NLP tasks. This process helps in focusing on more meaningful words that significantly contribute to understanding the context and content of the text, thereby facilitating improved performance in tasks like sentiment analysis and text classification. Stop word removal can be removed using various libraries. Two of the most commonly used are NLTK (Natural Language ToolKit) and Spacy. For **NLTK**, the `"nltk.corpus.stopwords.words('English')"` is used to remove the stopword for English. NLTK support various languages, just replaced the 'English' with the desired language, and it is supported by NLTK, such as "German", "dutch", and "Arabic". (Deepanshi, 2024) For **spaCy**, first, it needs to load the language model, for example, English (en) using `"spacy.load('en_core_web_sm')"`. Once loaded, the stop words can be accessed via `"nlp.Defaults.stop_words"`, Where the nlp is the loaded language model. (Educative, n.d.)

3.1.1.2 Case Normalization

Case normalization, or converting all text to lowercase, is a fundamental technique in text normalization for natural language processing (NLP) that enhances data consistency, reduces complexity, improves accuracy, and mitigates noise.(Sarguroh, 2023) By ensuring that words such as "Apple," "apple," and "APPLE" are treated uniformly, lowercasing helps maintain consistent recognition across texts, regardless of formatting. This simplification reduces the number of unique tokens, making learning processes more efficient for models, and standardizes inputs, which is crucial for reliable performance in NLP tasks like sentiment analysis and topic classification. Additionally, it helps smooth out formatting discrepancies from diverse data sources, thus minimizing noise and potential inaccuracies in the analysis.(Nabil, 2023) **Python** use `“.lower()"` built in string method to convert text to lowercase (Alabi, 2022) and **Pandas** use `“.str.lower()"` to convert text columns to lowercase. (Mishra, 2022)

3.1.1.3 Punctuation Removal

Punctuation removal is crucial in text processing to ensure consistency and improve the accuracy of algorithms by eliminating extraneous characters that do not contribute to the text's semantic meaning. This step is particularly important in natural language processing (NLP) tasks like tokenization, where punctuations can interfere with the accurate segmentation of words, or in sentiment analysis, where the focus needs to be purely on the textual content rather than formatting.(MistrY, 2024)

Various Python libraries provide built-in functions to assist with this process. For example, the Pandas library allows for easy manipulation of text data within DataFrames using the `.str.replace()` method (W3Schools.com, n.d.), which can efficiently remove punctuation from strings in a DataFrame column. Similarly, Python's standard string library includes a punctuation constant that can be used in conjunction with list comprehension or regular expressions (from the `re` library) to filter out unwanted characters from strings. (GeeksforGeeks, 2023) These methods help streamline data cleaning and preparation, making the text data more amenable to further analysis and processing.

3.1.1.4 WhiteSpace Normalization

Whitespace normalization is a crucial process in text preprocessing that involves adjusting the whitespace between words and sentences to a uniform standard. This step ensures that variations in whitespace, such as multiple spaces, tabs, or new lines, do not affect the analysis of text data, such as during tokenization or when performing word count operations. Uniform whitespace helps in maintaining consistency across text data, which is essential for accurate NLP tasks and data comparison. (Singh, 2023)

1. **Python's str library:** The `str.split()` function can be used to divide a string into a list of words, ignoring any extra spaces, and `str.join()` can then be used to concatenate these words with a single space between each. This effectively normalizes all whitespace to a single space. For example, the string 'hello world' becomes 'hello world'. (W3Schools.com, n.d.-b)
2. **Pandas library:** Within a DataFrame, the `.str.replace()` method can be used to substitute multiple whitespace characters with a single space using a regular expression. The regex pattern `'\s+'` matches one or more whitespace characters, which can be replaced with a single space `' '`. This method is highly effective in data cleaning operations involving large datasets typically stored in DataFrames. (*pandas.Series.str.replace* — Pandas 2.2.2 Documentation, n.d.)

These methods ensure that data remains consistent and easily comparable, removing any discrepancies caused by formatting differences in whitespace. This kind of normalization is particularly useful in preparing text for tasks like machine learning modeling, where input consistency can significantly impact the performance and outcomes of algorithms.

3.1.1.5 Lemmatization

Lemmatization is a natural language processing technique that reduces words to their lemma or canonical form, ensuring that variations related to time, quantity, or gender are normalized to a base form. This process is more syntactically sensitive than stemming, maintaining the grammatical category of words while standardizing them. For example, "girls" and "boys" become "girl" and "boy," respectively, and "chatting" reverts to "chat." Lemmatization proves particularly useful in applications like sentiment analysis for social media, where maintaining accurate semantic and syntactic representations enhances model performance. It utilizes rule-based or corpus-based methods, with the former applying grammatical rules and the latter relying on a tagged corpus to provide accurate lemmas for each part of speech. This precision is crucial for tasks requiring nuanced language handling, such as building knowledge graphs or enhancing machine learning models, making lemmatization a valuable tool in advanced NLP applications.(Duque, 2021)For **NLTK**, the "**wordNetLemmatizer**" class is used for lemmatization. **Spacy** uses "**[token.lemma_ for token in doc2]**" to perform lemmatization where "doc2" is the string that needs to perform lemmatization.

3.2 Tokenization

Tokenization is a critical technique in Natural Language Processing (NLP) that involves segmenting text into smaller parts called tokens, which can range from characters to words, facilitating easier analysis and understanding by machines. For example, the sentence "Chatbots are helpful" can be word-tokenized into ["Chatbots", "are", "helpful"], illustrating a straightforward approach where spaces dictate token boundaries. Alternatively, character tokenization would break it down further into individual letters, providing a granular view useful for languages without clear word separations or for specific tasks like spelling correction. Subword tokenization strikes a middle ground, breaking words into meaningful chunks, such as "Chatbots" into ["Chat", "bots"], which is beneficial for handling morphologically rich languages or out-of-vocabulary challenges in NLP tasks. Tokenization supports crucial applications like search engines, which parse queries into tokens to retrieve relevant results, and machine translation, where tokenized text is translated and reassembled to maintain the original context. However, tokenization faces challenges such as ambiguity in language interpretation, handling languages without clear word boundaries, and managing special characters in texts like email addresses. Advanced tokenization methods like the **BERT tokenizer**, which is context-aware, and tools like **NLTK** and **Spacy** offer sophisticated functionalities to navigate these complexities, enhancing text processing and thus improving the performance of NLP models.(Abid Ali, 2023) Bert Tokenizer use the "**BertTokenizer.from_pretrained('bert-base-uncased')**" to load the pre-trained bert Tokenizer to split the input text into tokens. (Kotamraju, 2022) The

"**word_tokenize**" function from the "**nlk.tokenize**" module in **NLTK** splits text into words based on spaces and punctuation. This simple yet effective method is suitable for basic tokenization tasks when complex models like BERT are unnecessary.(GeeksforGeeks, 2017) For **Spacy** it use "**spacy.load("en_core_web_sm")**" to load the english language model; after that the "**nlp('variables')**" is used to process the text with the model where the "Variables" is the text being processed and then use the "**[token.text for token in doc]**" to extract the text for each token. (GeeksforGeeks, 2022)

3.2.1 Vectorization

After tokenization, **TF-IDF (Term Frequency-Inverse Document Frequency)** is applied to convert text into a numerical format that machine learning algorithms can process, bridging human language and machine interpretation. This conversion is essential for allowing computers to effectively analyze and derive insights from large volumes of text data. The main reason to select TF-IDF instead of other Vectorization techniques such as Count Vectorization, Bag of Words, Word Embeddings(Word2Vec and GloVe), and Contextual Embeddings(BERT and GPT) is that it is computationally effective considering that the project needs to compare with a massive volume of data (Job description) thus this vectorization technique is selected. Even though the help of a tag can significantly reduce the number of job descriptions to be compared to, the processing speed is still crucial in this case, as not all job seekers have the patience to wait for the result to be displayed if it is taken too long most likely the user would change to a different platform.

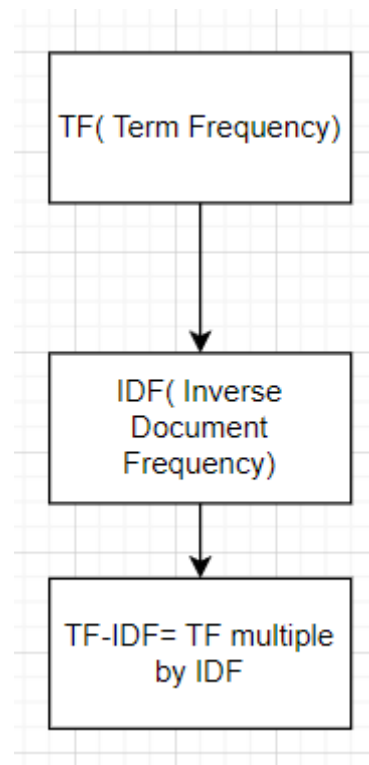


Figure 14 Flow of How TF-IDF is calculated.

TF-IDF (Term Frequency-Inverse Document Frequency) is a crucial statistical technique in Natural Language Processing (NLP) that assesses the significance of a word within a document in relation to a broader corpus, helping to determine its relevance by considering the term's frequency in the document and its rarity across all documents. It is especially useful for extracting meaningful information from unstructured text by assigning higher weights to terms that are frequent in specific documents but less common overall. This method is instrumental in various NLP applications, including search engines for ranking document relevance to queries, text classification to identify key features, information extraction, keyword extraction, recommender systems, and sentiment analysis, where it helps pinpoint crucial words contributing to sentiment. While highly valuable, TF-IDF has limitations such as ignoring word context and relationships, sensitivity to stop words, and challenges with large vocabularies, which can affect its performance in complex linguistic tasks. Despite these challenges, TF-IDF remains a foundational tool in NLP, widely applied for its ability to effectively handle large text corpora and provide interpretable results across multiple languages.(Pradeep, 2024)

Consider a term that appears 20 times within a document consisting of 100 total words. The Term Frequency (TF) can be calculated as below, Where the 20 indicates the number of times the term appeared in the document and is divided by the total number of the terms in the documents.

$$TF = \frac{20}{100} = 0.2$$

Figure 15 Numerical example of TF formula applied.

Suppose there is a collection of 10,000 documents, and a specific term appears in 100 of these documents. The Inverse Document Frequency (IDF) can be calculated as below, where the 10000 indicate the number of documents in the corpus and 100 indicate number of documents in the corpus contain the term.

$$IDF = \log \frac{10000}{100} = 2$$

Figure 16 Numerical example of IDF formula applied.

It's important to note that there are various methods for calculating the IDF score in text analysis. Commonly, the base 10 logarithm is employed, but some libraries might opt for using the natural logarithm instead. Additionally, to prevent division by zero, a value of one is often added to the denominator during the calculation, ensuring that terms which appear in all documents still yield a finite IDF score. (Figure 10)

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term} + 1}\right)$$

Figure 17 Another Formula of IDF

The TF-IDF score can be calculated by multiplying the TF score (0.2) and IDF score (2) and get the value 0.4.

$$TF-IDF = 0.2 * 2 = 0.4$$

Figure 18 Numerical example of TF-IDF formula applied.

3.3 Named Entity Recognition (NER) and Cosine Similarity

3.3.1 Named Entity Recognition (NER)

3.3.1.1 What is NER and How it work

Named Entity Recognition (NER) is a crucial technique in natural language processing that identifies and categorizes key information in text into predefined categories such as Person, Location, Organization, or custom categories like Healthcare Terms. NER works by extracting

entities and evaluating their relevance based on frequency and uniqueness within a document or across a corpus. This technology has a wide range of applications including enhancing customer support by quickly categorizing communications, aiding healthcare professionals by extracting key data from reports, optimizing search engine results, and streamlining HR processes by summarizing CVs efficiently. Methods for implementing NER vary from simple dictionary-based approaches to more complex rule-based and machine learning methods, with the latter offering the ability to recognize variations in entity names. Advanced deep learning techniques further improve accuracy by understanding the relationships between words. NER can be effectively implemented using tools like spaCy, which provides built-in NER capabilities and different models optimized for various levels of detail, and the Stanford NER tagger, known for its robustness in identifying a wide range of entity types through different model configurations. (A Comprehensive Guide to Named Entity Recognition (NER), 2022)

3.3.1.2 How to Apply NER and Why it matter in this Project

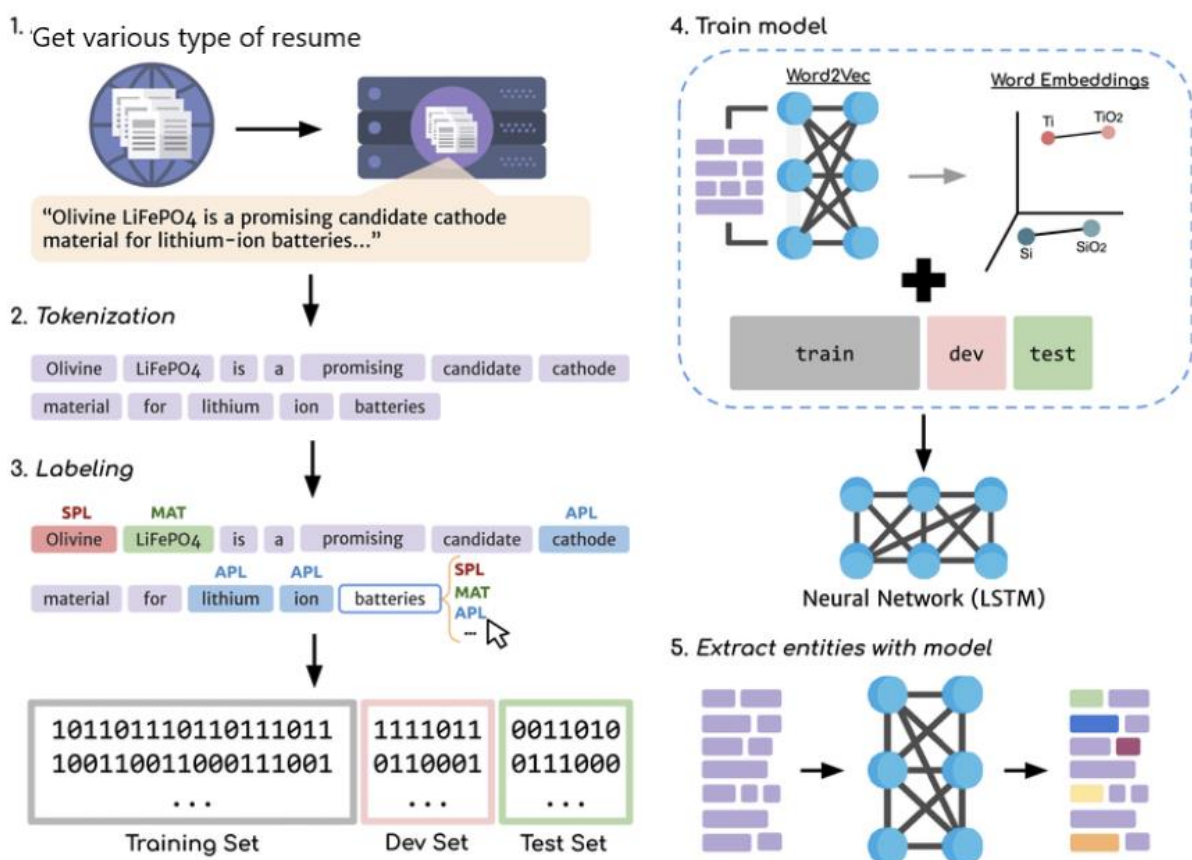


Figure 19 Work Flow of NER

Before applying the NER to extract the resume information, the best model needs to be selected to capture the information correctly; thus, it first needs to get the resume dataset. Clean the dataset, Such as standardising the text format and removing the typo to ensure uniformity. After that, tokenise the text into smaller units so it can be processed later on more quickly. The dataset

must be annotated for NER, meaning each named entity in the text is labelled according to its category (e.g., person, organisation, location, etc.). The annotation process can be manual or automated with the help of pre-existing tools. Manual annotation: Involves human annotators labelling the text. This is time-consuming but often necessary for achieving high-quality annotations. Automated annotation Can be done using existing NER tools to pre-label a dataset, which is manually corrected. This method can speed up the annotation process but might require adjustments to fit specific needs. After that, select the Machine learning model that can be used for NER tasks such as Conditional Random Fields (CRF), Support Vector Machines (SVM), and Recurrent Neural Networks(RNN). After that, the annotated dataset and extracted features were added to the machine learning and pre-trained models (NLTK, SpaCy, and the Stanford NER) to learn patterns and relationships between words and their corresponding entity labels. The model evaluation is done to measure the performance of the NER model using precision, recall, accuracy, and an F1 score. In the end, choose the best model from there and then use it to capture the data from the resume. (Agata, 2023)

When applying the best model on a resume uploaded by the user, It should be used after the tokenisation so the model can capture the data correctly. NER are essential In this project as resumes are dense and contain varied information in a non-standard format; NER can help to identify and classify the critical components of the data (Names, Skills, Education) so the employer can look at the essential information instead of reading the entire resume; this can help the employer to save valuable time and money. Another example that NER can facilitate quicker and more efficient candidate evaluation is if a recruiter is looking for a candidate who graduated from "TARUMT" with a degree in "Computer Science", NER can identify the educational background in each resume and pull out the necessary information that matches the criteria.

3.3.2 Cosine similarity

Cosine similarity is a metric that gauges the similarity between documents by measuring the cosine of the angle between two vectors in a multi-dimensional space, representing the word counts of each document. Unlike simpler methods that count the number of common words or use Euclidean distance—which can misleadingly suggest similarity based on sheer document size or word frequency—cosine similarity effectively captures the orientation rather than the magnitude of the vectors. This means that even if two documents vary significantly in length or word count (for example, if "cricket" appears 50 times in one document and 10 times in another), they can still be considered similar if they have a smaller angle between them, indicating closer thematic or contextual alignment. This approach overcomes the inherent limitations of merely counting common words or relying on Euclidean distance, providing a

more accurate measure of true document similarity.(Prabhakaran, 2022) Figure 13 indicates that the “Hello World!” and “Hi World!” where contribute a smaller angle which indicates higher cosine similarity.

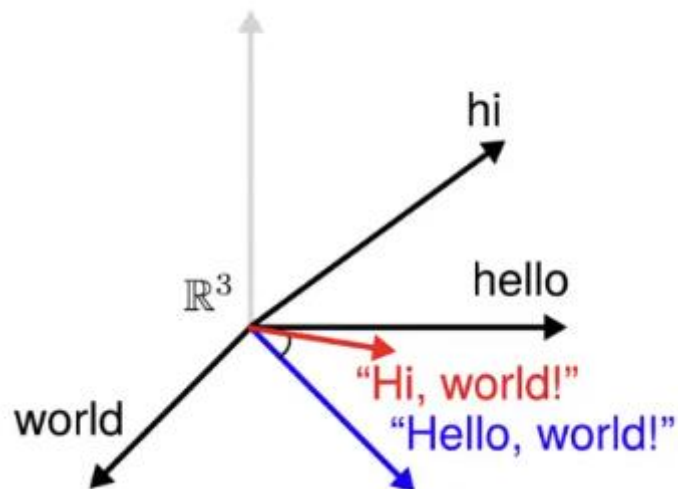


Figure 20 Example of cosine of the angle between two vectors in a multi-dimensional space. As the cosine similarity measurement gets closer to 1, then the angle between the two vectors A and B is smaller. The images below depict this more clearly.

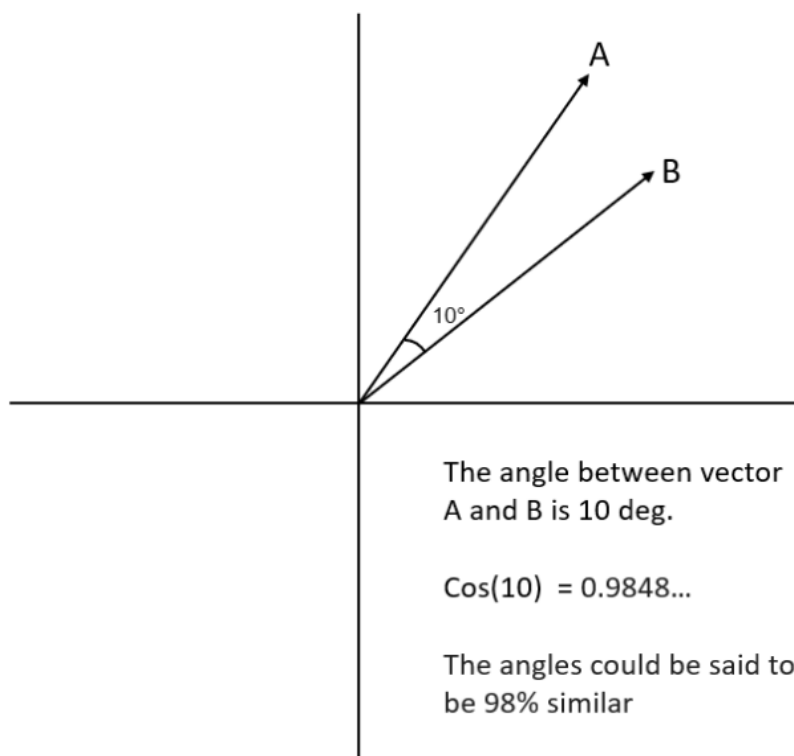


Figure 21 Example of high cosine similarity

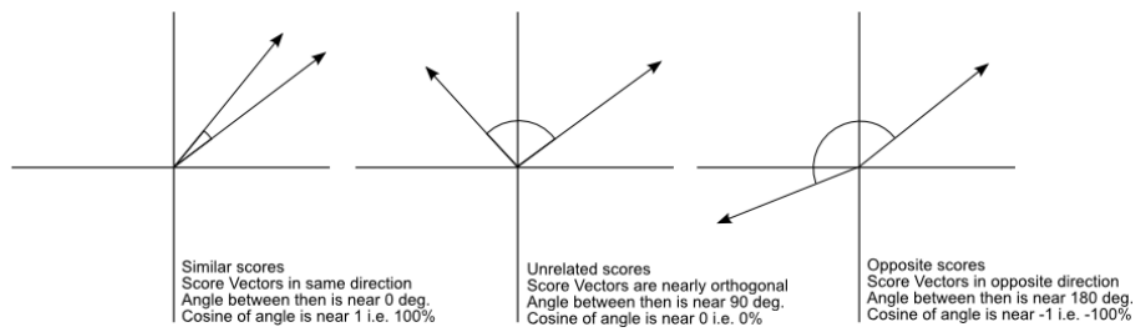


Figure 22 The Cosine Similarity values for different documents, 1 (same direction), 0 (90 deg.), -1 (opposite directions).

The Similarity and distance between two vector are inversely related. Where the similarity decrease (toward -1), the distance between two vector increase (Great euclidean distance). (Refer to figure 16) A high cosine similarity (close to 1) means a smaller Euclidean distance (smaller distance between two vector) (refer to figure 17)

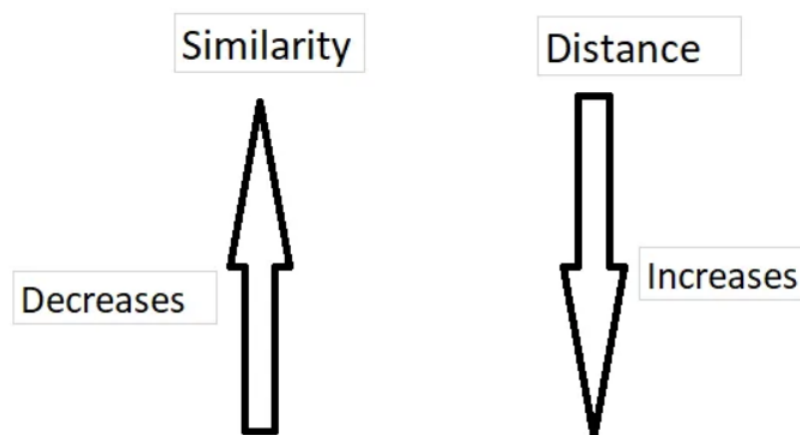


Figure 23 Similarity decrease when distance between two vector increase.

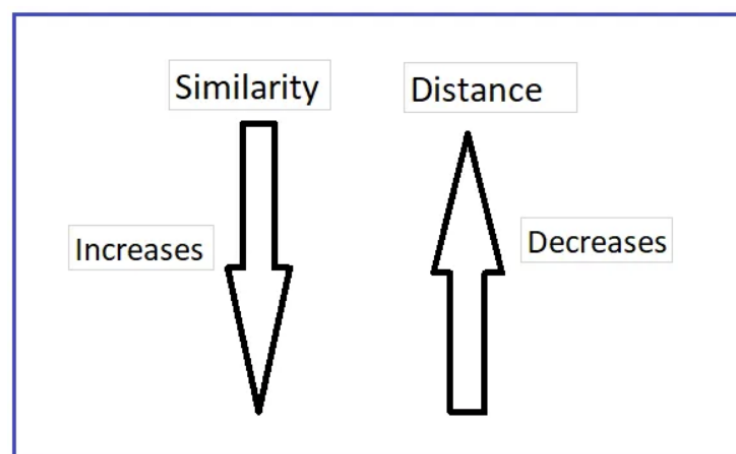


Figure 24 Similarity increase when distance between two vector decreases.

The cosine similarity between these vectors is calculated using a formula where the numerator is the dot product of A and B , and the denominator is the product of their magnitudes. This formula effectively quantifies the cosine of the angle between the two vectors, yielding a range from -1 to 1. A cosine similarity value of 1 means the vectors are identical in orientation, 0 denotes orthogonality, indicating no similarity, and -1 suggests that the vectors are entirely opposed.

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$

Figure 25 Cosine similarity Formula

In job recruitment, cosine similarity can be used to compare the text of a resume with the text of a job description. A higher cosine similarity score indicates a smaller angle and, thus, a more remarkable similarity between the two documents. High cosine similarity is a great indicator of matching the job description and resume because, For example, A job description requiring expertise in Python, SQL, and data visualization will form a vector emphasizing these skills. A candidate whose resume highlights extensive experience with Python, SQL, and data visualization tools like Tableau will align well with the job vector, resulting in a high cosine similarity score. A high cosine similarity score suggests that the resume's content is aligned with the keywords and skills outlined in the job description. This means the candidate's experience and skills are likely a good match for what the job requires. In the recruitment project proposed by the team where the job description will be compared with the resume uploaded by the job seeker, it first will give some authority to the candidate where they can know which job is more likely to align well with their strength, saved their time on job hunting process. Most importantly, they can know which job has a higher chance of success; thus, it can reduce the likelihood of getting rejected. On the other hand, the employer will most likely receive a better-quality resume. Without this, candidates would not know which job better fits them. As a result, they might apply all the jobs that align with their expertise.

3.4 Data storage

MongoDB is widely acclaimed for its flexibility, scalability, and exceptional performance with large volumes of data, making it a popular choice in various domains

such as big data applications, content management, real-time analytics, and more. Its schema-less nature allows for storing unstructured data like resumes, where each document can vary in structure, accommodating diverse candidate details without complex joins or transactions. This flexibility ensures quick data retrieval and easy updates—essential features for dynamic environments like job portals where new resumes and job descriptions are continuously uploaded and updated.(MongoDB, n.d.)

The document-oriented setup of MongoDB facilitates rapid development and deployment, as it mirrors the object-oriented programming structure, thereby reducing the impedance mismatch between the database and application code. Moreover, its inherent scalability, supported by sharding and load balancing, makes it adept at handling growth in data volume and user load, ensuring that the system remains performant and responsive as the user base expands.(Gillis & Botelho, 2023)

For a resume handling system, MongoDB's powerful querying capabilities offer an advantage; for instance, its full-text search can streamline the matching of job seekers with relevant positions by efficiently searching through extensive text fields in resumes and job descriptions. Additionally, its aggregation framework can perform complex processing directly in the database, reducing the application's workload.

Given these strengths, MongoDB is a fitting choice for projects focused on managing and querying large datasets like resumes and job listings efficiently. Its ability to handle high volumes of data with speed and agility aligns well with the needs of job matching systems, ensuring that the system can scale and adapt as user demands and data volumes grow.

3.4.1 Employer

When the Employer posts the job listing and the job description to the platform, it is stored in an online cloud storage system such as Google Cloud, Dropbox, IBM, elastic search, MongoDB atlas or OneDrive to store the original resume file.

Chapter 4

System Design

4 System Design

This chapter details the workflow, preprocessing steps, and user interfaces involved in the job-matching platform. It covers the process from resume upload to job matching, explaining the conversion of various resume formats to TXT, data preprocessing, and application of Named Entity Recognition (NER) and Term Frequency-Inverse Document Frequency (TF-IDF) for semantic analysis. The storage and retrieval of processed data in MongoDB are discussed, along with the use of cosine similarity for matching job descriptions with resumes. The user interface for both employers and job seekers is also described, highlighting features such as login, job submission, resume management, and job search functionalities. This comprehensive overview provides a clear understanding of the system's workflow, technologies used, and user interactions.

4.1 System Flow

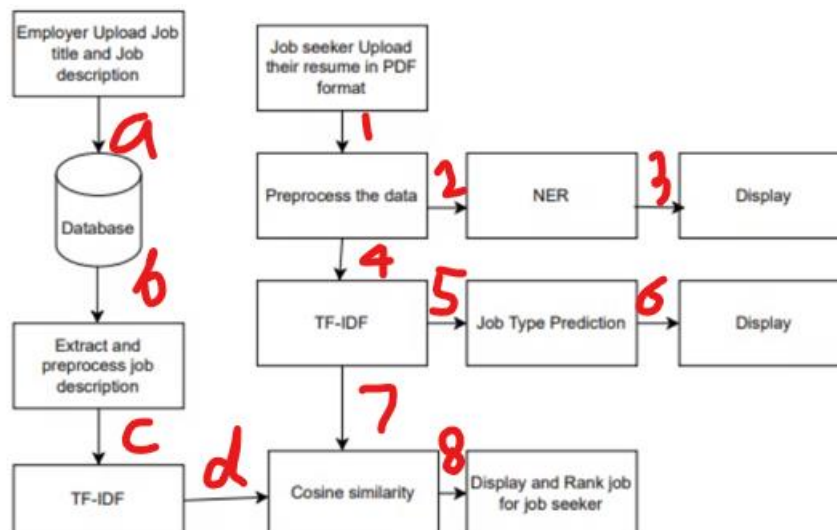


Figure 26 System design Work flow.

4.1.1 Job Seeker upload their resume (1)

This function extracts text from uploaded PDF files using **pdfplumber**, which is crucial for processing resumes. By converting PDF content into plain text, the system can further preprocess and analyze the data, laying the groundwork for all subsequent text processing tasks.

4.1.2 Preprocess the Data (2)(4)(C)

Once job seekers upload their resumes/Job Descriptions extracted from the database, the data undergoes a series of preprocessing steps to prepare it for analysis. The `.lower()` method

converts all text to lowercase to ensure uniformity and reduce case sensitivity. The `re.sub()` function removes non-alphanumeric characters and normalises whitespace, which strips away punctuation and unnecessary spaces to prevent errors during word tokenization. The `split()` method then breaks the cleaned text into individual words, allowing for a separate analysis of each word. Words are lemmatized using a lemmatizer to reduce them to their base forms, making the text more consistent across different grammatical variations. Concurrently, stop words are removed to reduce data volume and focus analysis on more meaningful content. These preprocessing steps are essential for transforming raw resume text into a standardized format, enhancing the effectiveness of subsequent analytical tasks and improving the accuracy of the system's outputs.

4.1.3 Named Entity Recognition for job description and resume(3)

Using spaCy default pre-trained model for Named Entity Recognition (NER) right after cleaning the data and before applying TF-IDF is a strategic choice in the text processing workflow for several reasons. After cleaning the data—which includes steps like removing stopwords, normalising case, and lemmatisation—the text is in a simplified and standardised form. This makes it an ideal state for NER, as cleaner text improves the accuracy of entity recognition. SpaCy's NER is particularly effective because it is trained on diverse texts, allowing it to accurately identify and categorise entities such as “Name” and “Language”

Positioning NER at this stage in the workflow ensures that the text is free from essential syntactic noise and semantically enriched, making the application of TF-IDF or any other statistical analysis more meaningful and targeted. This sequence optimises the text data for deeper analysis and more insightful outcomes in any NLP pipeline.

4.1.4 TF-IDF for resume and job description(5) (7)

Implementing TF-IDF at this stage allows for quantifying how important these identified entities are within the document relative to the entire corpus, thus emphasising their significance. This method ensures that the vector space model, used subsequently in machine learning models or other analyses, reflects the actual informational value of the text, enhancing the accuracy and relevance of outcomes in tasks like information retrieval and text classification. This strategic placement optimises the data for further analytical tasks by focusing on the contextual and semantic richness of the text.

4.1.5 Job Type Prediction (6)

In the system design, after a job seeker uploads their resume and it undergoes cleaning and TF-IDF vectorization, the next step is the prediction phase, which is strategically placed at this stage to ensure the model operates on well-structured and meaningful data. Placing the prediction step here, rather than earlier, allows the model to use clean and representative features extracted through TF-IDF, rather than raw, noisy text data, which enhances prediction accuracy. Likewise, placing it after TF-IDF and not later ensures that the insights derived are immediate and actionable without unnecessary delays or additional processing. The model used for prediction is stored in a pickle file (`tuned_logistic_regression_model.pkl`), allowing the pre-

tuned and best-performing logistic regression model to be quickly loaded and reused without the need to retrain from scratch each time. This file is accessed and loaded using Python's pickle module, which efficiently handles serialization and deserialization of Python objects. The `predict_proba` method generates the probabilities for all classes, and `predict` method determines the most likely class for the input resume, ensuring a swift and effective matching of resumes to potential job openings.

4.1.6 Similarity Rank

Cosine similarity is a crucial metric used in the resume screening system to match job seekers with relevant job openings based on the content of their resumes and the job descriptions provided by employers. It is used to calculate the similarity between two text documents, in this case, the TF-IDF vector of a resume and the TF-IDF vector of a job description.

The reason cosine similarity is applied after the TF-IDF transformation of both resumes and job descriptions, and not before, is because TF-IDF (Term Frequency-Inverse Document Frequency) provides a weighted representation of word counts, emphasizing words that are important and unique to a document. This weighting helps mitigate the issue of common but less significant words overwhelming the analysis, which could happen if raw text or simple term frequencies were used.

By converting the text data of both resumes and job descriptions into TF-IDF vectors before applying cosine similarity, the system can more accurately measure how closely related the content of a resume is to the content of a job description. This comparison is based on the angle between the two vectors in a multi-dimensional space—the smaller the angle, the greater the similarity.

Using only the job description rather than the entire job posting or other extraneous information ensures that the similarity measurement focuses solely on the essential requirements and responsibilities expected from the candidate. This focus helps in efficiently matching job seekers with positions that align closely with their skills and experiences, thus enhancing the relevance of job recommendations provided by the system.

4.1.7 Database

Using MongoDB to store user credentials, job descriptions, and job details offers strategic advantages for the project, particularly in optimizing the job matching process. MongoDB, being a flexible NoSQL database, is well-suited for handling large volumes of unstructured data, such as job listings and related details, which can vary in format and complexity. It is used

to store user credentials securely by hashing passwords with bcrypt before saving them to ensure that sensitive information is protected. Additionally, job descriptions and other related data are stored, allowing for easy retrieval and manipulation during job filtering and matching processes. MongoDB's scalability ensures that it can handle increasing amounts of data as the system grows, providing reliable performance even with a larger user base and job postings. Its dynamic schema design supports efficient updates, while its real-time querying capabilities enable users to search for jobs quickly and effectively, making it a powerful tool for the backend management of a job matching system. These features, combined with MongoDB's ease of integration with other technologies, make it an essential component in ensuring the smooth operation and security of the platform.

4.2 User Interface

4.2.1 Employer

4.2.1.1 Login Register Page

The diagram illustrates a user interface for an employer's login and registration page. It is enclosed in a large rectangular frame. The interface is divided into two main sections: 'Login' and 'Register'.

The 'Login' section is located in the upper half of the frame. It begins with the label 'Login' in a bold font. Below this label are two text input fields: the first is labeled 'User Name' and the second is labeled 'Password'. Both labels are in a bold font. Below the 'Password' input field is a button labeled 'Login' in a bold font.

The 'Register' section is located in the lower half of the frame. It begins with the label 'Register' in a bold font. Below this label are two text input fields: the first is labeled 'User Name' and the second is labeled 'Password'. Both labels are in a bold font. Below the 'Password' input field is a button labeled 'Register' in a bold font.

Figure 27 Login, Register page for Employer

For employers, the Job Matcher application features a robust registration and login page that is vital for securely managing candidate information and recruitment processes. Registration is necessary to create a unique employer account, ensuring that each account is distinct, with no duplicates in the system. This uniqueness is enforced by the database, which prevents the creation of two accounts with the same credentials. Once registered, employers can log in to authenticate their identity, which is essential for accessing resumes securely and managing recruitment tasks. This secure authentication helps maintain the confidentiality and integrity of applicant data, safeguarding against unauthorized access and ensuring compliance with privacy regulations. Furthermore, the authenticated environment allows employers to efficiently track applicant statuses, manage job postings, and make informed hiring decisions, thereby enhancing the overall recruitment workflow and maintaining data security.

4.2.1.2 CRUD Function on employer Page

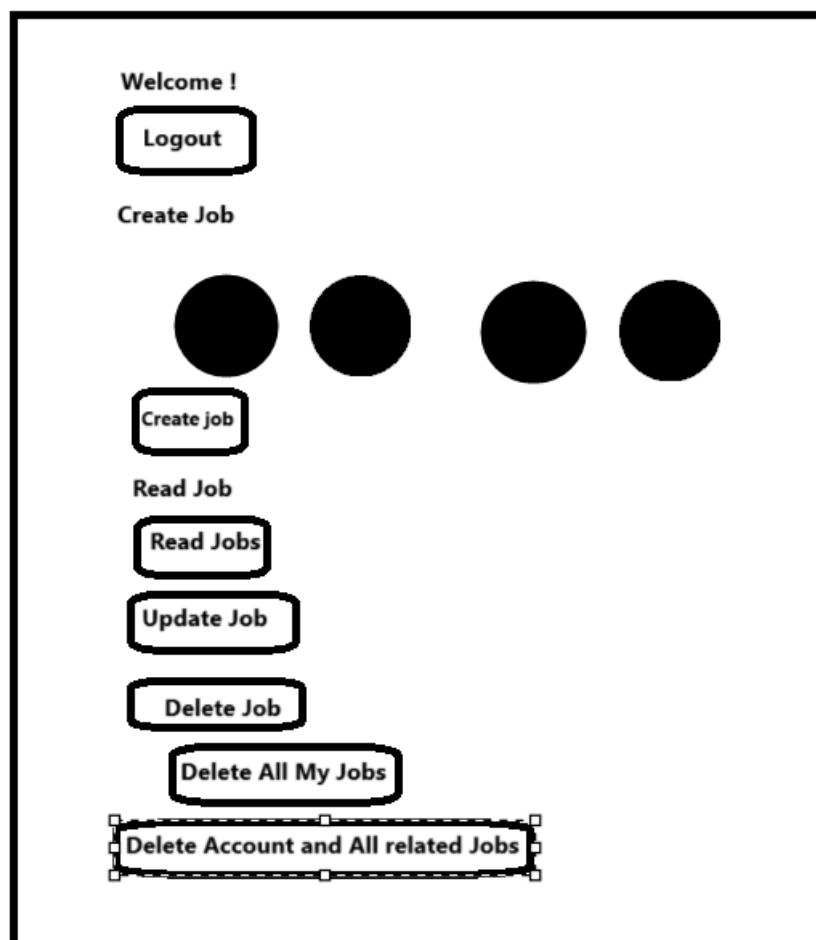


Figure 28 Employer page after login

In the employer interface of the Job Matcher application provides a robust platform for employers to manage job postings. This segment of the application includes various functionalities tailored for comprehensive job management. Employers can log out securely using the logout button, ensuring that their session ends properly to maintain security. The interface allows employers to create new job listings with essential details, view existing job listings, and update or delete specific jobs as required. There is also an option to delete all jobs posted by the employer in one action, streamlining the process for removing outdated or filled positions. Additionally, for ultimate account management, employers can choose to delete their entire account along with all related job listings, ensuring data privacy and leaving no residual information in the system. This suite of tools is designed to facilitate easy and efficient management of recruitment processes, enhancing the employer's ability to handle their recruitment needs effectively within the platform.

4.2.2 Job seeker

4.2.2.1 Home /Submit Resume Page

Upload Resume

Display Resume

Extracted Skills and information

Submit

Figure 29 Page for job seeker to submit resume to the system

In the "Submit Resume" section of the job application portal, job seekers are provided with a streamlined interface to upload their resumes. This section is designed to accept resume uploads, after which the resume is displayed on the screen for the applicant's review. Upon uploading, job seekers can initiate the processing of their resume by clicking a designated button. This action triggers several automated processes: the application cleans the data, applies TF-IDF vectorization for text analysis, and uses machine learning to predict the job type that best matches the resume's content. Additionally, the system extracts vital information such as the candidate's phone number, email address, name, and languages spoken directly from the resume. This comprehensive processing ensures that the data is well-structured and enhances the matching accuracy between job seekers' profiles and available job listings, simplifying subsequent steps in the job application process.

4.2.3 Filter Job Listing

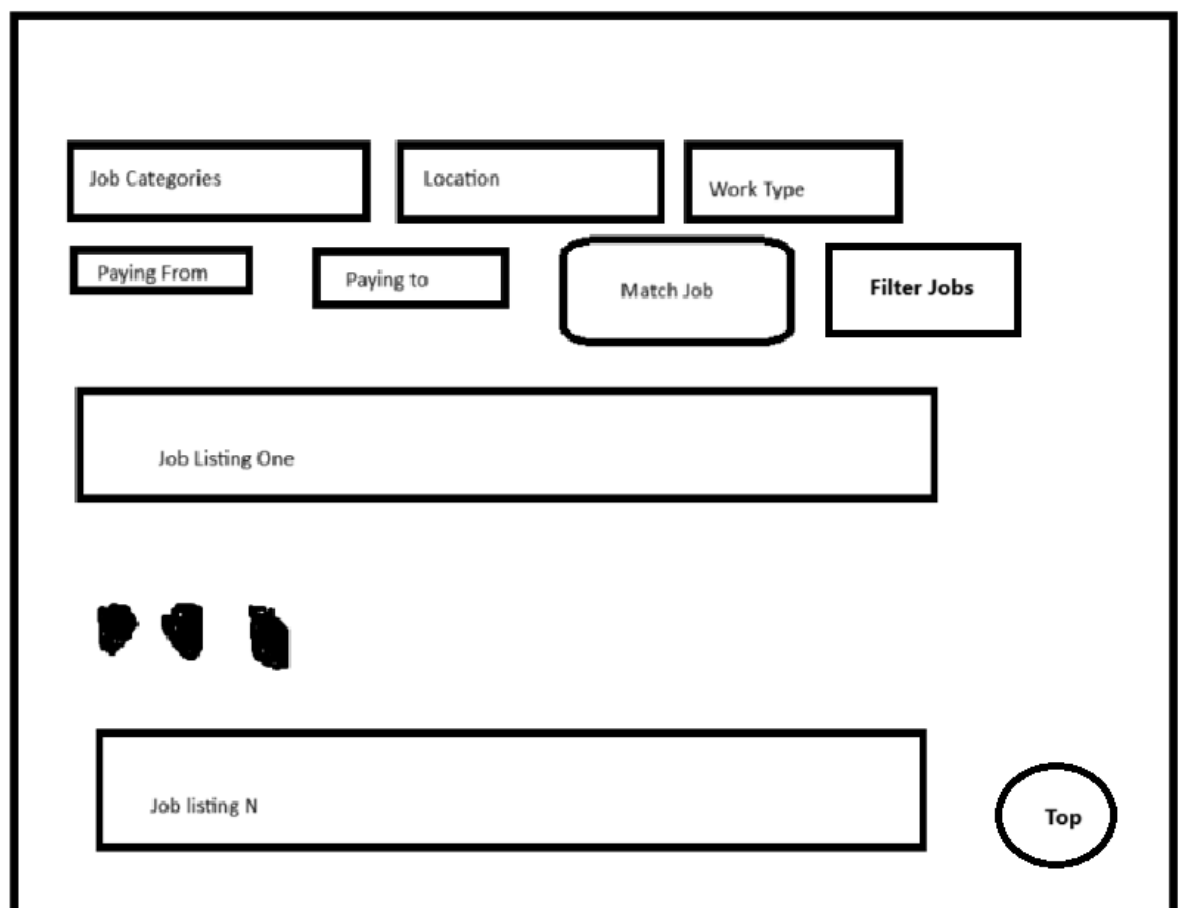


Figure 30 All the job listing available for Job Seeker

In the "Filter Job Listings" section of a job search platform, users can refine their search for suitable positions through a comprehensive filtering system that includes options for Tags, Location, Work Type, Job Categories, and Pay Range. This feature enhances the user experience by allowing job seekers to tailor their searches to match their specific skills,

preferred work settings, and salary expectations. After setting the desired filters, candidates can utilize the 'Filter Jobs' button to narrow down the list of job postings according to their specified criteria. Additionally, there is a 'Find Best Match' button which, when clicked, ranks the jobs based on their relevance to the user's profile using cosine similarity—a method that assesses the similarity between the job description and the candidate's resume. For added convenience, a 'Back to Top' button is strategically placed at the bottom right of the page, enabling users to easily navigate back to the top of the listings after an extensive search. This multifaceted approach not only streamlines the search process but also increases the likelihood of finding the most fitting job opportunities.

4.2.4 Visualization

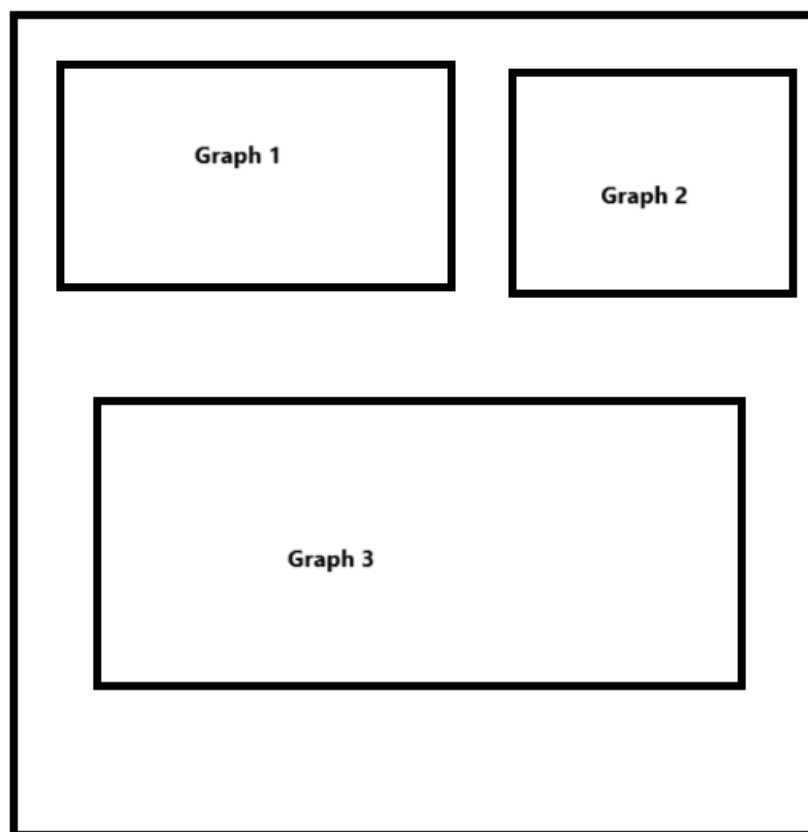


Figure 31 Visualization Page

The "Visualization" section of the job search platform offers a dynamic and interactive experience for both employers and job seekers by displaying various graphical representations of job data. This feature pulls information directly from the jobs posted by employers, presenting it in a series of graphs that can be customized based on user preferences. Employers and job seekers can interact with the data through filters that adjust what is displayed, such as job categories, employment type, and salary ranges. The available graphs may include bar

charts comparing the number of jobs by category, pie charts showing the distribution of job types, and line graphs tracking job posting trends over time. Each graph is designed to be interactive, allowing users to click on elements to drill down for more detailed views or to adjust parameters to see how different filters impact the data presented. This visualization tool not only enhances the understanding of job market dynamics but also aids in strategic decision-making by highlighting trends and gaps in the job market.

Chapter 5

Implementation and Testing

5 Implementation and Testing

This chapter delves into the practical implementation aspects of the job-matching system, detailing the construction and functionality of various user interfaces, including the main user interface, employer interface, and job listings. It also covers the visualization features that enhance user interaction and the underlying code that supports these elements. The chapter also outlines the rigorous testing protocols applied to ensure the system's reliability and effectiveness, with comprehensive test cases for each component. Furthermore, it discusses the database setup and the security measures implemented to safeguard user data, ensuring a robust and secure environment for handling sensitive information.

5.1 Implementation

This Section provides a detailed examination of the implementation process for the job-matching system, covering the code that powers the main user interface, employer interface, job listings, and visualization components. Each Section meticulously describes the functionalities embedded within these interfaces, from user interactions to data handling and display mechanisms. The system's efficiency in job matching and user engagement is illustrated, instilling confidence in its performance.

Main User Interface

5.1.1.1 PDF Text Extraction

```
def read_pdf_file(pdf_file):  
  
    with pdfplumber.open(pdf_file) as pdf:  
  
        return " ".join(page.extract_text() for page in pdf.pages if page.extract_text())
```

Explanation:

This function extracts text from uploaded PDF files using **pdfplumber**, which is crucial for processing resumes. By converting PDF content into plain text, the system can further preprocess and analyze the data, laying the groundwork for all subsequent text processing tasks.

5.1.1.2 Text Preprocessing

```
def process_text(text):  
  
    text = text.lower()  
  
    text = re.sub(r'^a-zA-Z0-9\s', '', text)  
  
    text = re.sub(r'\s+', ' ', text).strip()  
  
    words = text.split()
```

```
cleaned_words = [lemmatizer.lemmatize(word) for word in words if word not in
stop_words]

text = ' '.join(cleaned_words)

return text
```

Explanation:

This function prepares the extracted text for analysis by converting it to lowercase, removing non-alphanumeric characters, and eliminating stopwords. It ensures that the text data is standardized and clean, which is essential for accurate feature extraction and similarity analysis.

5.1.1.3 *Extracting and Displaying Contact Information*

```
def extract_contact_info(text):

    email_pattern = r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'

    emails = re.findall(email_pattern, text)

    phone_pattern = r'''

        (\+?\d{1,3}\s)?

        (\(\d{3}\))

        [\s.-]?

        (\d{3})

        [\s.-]?

        (\d{4})

    '''

    phones = re.findall(phone_pattern, text, re.VERBOSE)

    phones = [' '.join(filter(None, phone)) for phone in phones]

    return emails, phones
```

Explanation:

Purpose: This functionality enriches the user experience by providing immediate feedback about key information extracted from the uploaded resume, such as email addresses and phone numbers. This helps the user ensure that the resume content is correctly interpreted by the system.

5.1.1.4 Extract Name and Language

```
def extract_additional_info(text):  
  
    doc = nlp(text)  
  
    persons = [ent.text for ent in doc.ents if ent.label_ == "PERSON"]  
  
    languages = [ent.text for ent in doc.ents if ent.label_ == "LANGUAGE"]  
  
    return persons, languages
```

Explanation:

This function leverages a pre-trained NLP model to process the input text and identify entities categorized as "PERSON" and "LANGUAGE". This is used to ensure the data input was accurate.

5.1.1.5 Loading the vectorizer

```
def load_vectorizer():  
  
    vectorizer_path = 'C:/Users/ongka/OneDrive/Desktop/FYP Spare/vectorizer.pkl'  
  
    with open(vectorizer_path, 'rb') as file:  
  
        vectorizer = pickle.load(file)  
  
    return vectorizer
```

Explanation:

This function is crucial for the text processing capabilities of the system, as it loads a pre-trained TF-IDF vectorizer from a serialized pickle file. The vectorizer is applied universally to both resumes and job descriptions, ensuring that all text data undergoes the same feature extraction process. This standardization is vital for the performance of the system's matching algorithms, which rely on a consistent and uniform vector space to accurately compare and match text content. By utilizing this single vectorizer for different types of text inputs, the system maintains consistency in how text data is transformed, enhancing the reliability and accuracy of predictions made by the machine learning models. This uniform approach to text vectorization is essential not only for semantic similarity assessments between different documents but also for ensuring that the input to predictive models is as effective as during their initial training phase. The standardization provided by the TF-IDF vectorizer plays a pivotal role in the system's ability to perform sophisticated text analysis and matching operations efficiently and accurately.

5.1.1.6 Loading the Machine Learning Model and Making Predictions

```
# Load the model and predict

model_filename = 'C:/Users/ongka/OneDrive/Desktop/FYP
Spare/tuned_logistic_regression_model.pkl'

with open(model_filename, 'rb') as file:

    loaded_model = pickle.load(file)

probabilities = loaded_model.predict_proba(tfidf_matrix)

max_prob = max(probabilities[0])

predicted_label = loaded_model.predict(tfidf_matrix)[0]
```

Explanation:

This code snippet is responsible for loading a pre-trained logistic regression model and using it to make predictions about the job category based on the TF-IDF vectorized text of a resume. It loads the model, calculates the probability of each category, and determines the most likely category.

5.1.1.7 Displaying Job Category

```
def display_job_category(max_prob, predicted_label):

    label_mapping = {

        0: 'ACCOUNTING',

        1: 'ENGINEERING',

        2: 'INFORMATION-TECHNOLOGY'

    }

    threshold = 0.55

    if max_prob < threshold:

        st.markdown("<div      class='info-header'>Job      Category:</div>",
unsafe_allow_html=True)

        st.markdown("<div      class='no-info'>Unknown      or      Irrelevant</div>",
unsafe_allow_html=True)

    else:

        predicted_category = label_mapping[predicted_label]
```

```
st.markdown("<div          class='info-header'>Job          Category:</div>",
unsafe_allow_html=True)

st.markdown(f"<div          class='info-content'>{predicted_category}</div>",
unsafe_allow_html=True)
```

Explanation:

This function provides visual feedback by displaying the predicted job category. It uses a threshold to determine if the prediction is confident enough to be displayed. This enhances the user experience by helping job seekers understand how their skills and experience align with specific job categories.

5.1.2 Main User Interface

5.1.2.1 Register

```
def register_user(username, password):

    user = users_collection.find_one({"username": username})

    if user:

        st.error("Username already exists, please choose another.")

        return False

    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())

    users_collection.insert_one({"username":          username,          "password":
hashed_password})

    st.success('Registration successful! You can now login.')

    st.session_state['username'] = username

    st.session_state['logged_in'] = True

    return True
```

Explanation:

Function: Checks if the username already exists in the `users_collection` database. If the username does not exist, it proceeds with registration.

Security: During registration, user passwords are hashed using `bcrypt` before being stored in the database. This method prevents plaintext passwords from being saved or exposed, enhancing security.

User Feedback: Provides immediate feedback on the registration process. If the username is taken, it alerts the user to choose a different username; if registration is successful, it confirms the successful account creation.

5.1.2.2 Login

```
def login_page():  
    username = st.text_input("Username", key='login_username')  
    password = st.text_input("Password", type="password", key='login_password')  
    if st.button("Login"):  
        if check_credentials(username, password):  
            st.session_state['logged_in'] = True  
            st.session_state['username'] = username  
            st.success('Logged in successfully!')  
        else:  
            st.error('Incorrect Username or Password')
```

Explanation:

Function: Provides input fields for username and password. It verifies the user credentials against the MongoDB collection.

Security: Verifies hashed passwords during login using bcrypt, ensuring that plaintext passwords are neither stored nor transmitted. This secure verification protects user data and prevents unauthorized access.

Session Management: Updates the session state based on authentication status. If the credentials are verified, it sets the session to logged in, reflecting the user's login status across the platform.

User Experience: Enhances user experience by providing clear feedback. If the credentials are incorrect, it informs the user with an error message, and if login is successful, it welcomes the user with a success message.

5.1.2.3 Create Job

```
def create_job_page():  
    st.header("Create Job")
```

```
create_company = st.text_input("Business Name",
value=st.session_state.get('username', ''), disabled=True)

JobCategory = ["Accounting", "Engineering", "Information-Technology"]

create_JobCategory = st.selectbox("Job Category", JobCategory)

create_title = st.text_input("Job Title *")

create_Job_Description = st.text_area("Job Description *")

create_location = st.text_input("Address Line *")

create_city = st.text_input("City, Town or Region *")

states = ["Johor", "Kedah", "Kelantan", "Kuala Lumpur", "Labuan", "Melaka",
"Negeri Sembilan", "Pahang", "Penang", "Perak", "Perlis", "Putrajaya", "Sabah",
"Sarawak", "Selangor", "Terengganu"]

create_state = st.selectbox("State *", states)

create_Postcode = st.text_input("Postcode *")

WorkType = ["Full-time", "Part-time", "Contract", "Casual"]

WorkTypeSelection = st.selectbox("Work Type", WorkType)

create_salary_from = st.number_input("Salary From", min_value=1, step=1)

create_salary_to = st.number_input("Salary To", min_value=create_salary_from,
step=1)

create_email = st.text_input("Email *")

common_benefits = ['Health insurance', 'Dental insurance', 'Retirement plan',
'Stock options', 'Paid time off', 'Remote work options', 'Gym membership', 'Free
lunches', 'Childcare']

selected_benefits = st.multiselect("Select benefits and perks",
common_benefits)

create_company_Size = st.number_input("Company Size", min_value=1, step=1)

if st.button("Create Job"):

    if not (create_title and create_Job_Description and create_location and
create_city and create_state and create_Postcode and create_email):
```

```
        st.error("Please fill in all required fields.")

    elif create_salary_from > create_salary_to:

        st.error("Salary From cannot be greater than Salary To.")

    elif not re.match(r"^[^@]+@[^@]+\.[^@]+", create_email):

        st.error("Invalid email format.")

    else:

        job = {

            "Business Name": create_company,

            "Job Category": create_JobCategory,

            "Job Title": create_title,

            "Job Description": create_Job_Description,

            "Address line": create_location,

            "City": create_city,

            "State": create_state,

            "Postcode": create_Postcode,

            "WorkType": WorkTypeSelection,

            "Salary From": create_salary_from,

            "Salary To": create_salary_to,

            "Email": create_email,

            "Benefit": selected_benefits,

            "CompanySize": create_company_Size,

        }

        result = collection.insert_one(job)

        st.success(f"Job Created with ID: {result.inserted_id}")
```

Explanation:

The `create_job_page` function provides employers with a user-friendly interface to post new job vacancies. Employers can specify job details such as category, title, description, location, type of work, salary range, and benefits. The process is safeguarded by validations that ensure all

required fields are filled and that the input formats are correct, particularly for emails and salary ranges. This function enhances the system's usability by enabling precise job listings, which helps attract qualified candidates.

5.1.2.4 Update Job

```
def update_job():
    st.header("Update Job")

    job_id = st.text_input("Enter the MongoDB ID of the job you wish to update",
key="update_job_id")

    if job_id:
        try:
            job_details = collection.find_one({"_id": ObjectId(job_id)})
        except Exception as e:
            st.error("Invalid ID format. Please enter a valid MongoDB ObjectId.")
            return

        if job_details:
            create_JobCategory = st.selectbox("Job Category", ["Accounting",
"Engineering", "Information-Technology"], index=["Accounting", "Engineering",
"Information-Technology"].index(job_details['Job Category']),
key=f"job_cat_{job_id}")
            create_title = st.text_input("Job Title *", value=job_details['Job Title'],
key=f"job_title_{job_id}")
            create_Job_Description = st.text_area("Job Description",
value=job_details['Job Description'], key=f"job_desc_{job_id}")
            create_location = st.text_input("Address Line", value=job_details['Address
line'], key=f"job_loc_{job_id}")
            create_city = st.text_input("City, Town or Region", value=job_details['City'],
key=f"job_city_{job_id}")
            states = ["Johor", "Kedah", "Kelantan", "Kuala Lumpur", "Labuan",
"Melaka", "Negeri Sembilan", "Pahang", "Penang", "Perak", "Perlis", "Putrajaya",
"Sabah", "Sarawak", "Selangor", "Terengganu"]
            create_state = st.selectbox("State", states,
index=states.index(job_details['State']), key=f"job_state_{job_id}")
            create_Postcode = st.text_input("Postcode", value=job_details['Postcode'],
key=f"job_postcode_{job_id}")
            create_salary_from = st.number_input("Salary From", min_value=1, step=1,
format="%d", value=job_details['Salary From'], key=f"salary_from_{job_id}")
            create_salary_to = st.number_input("Salary To", min_value=1, step=1,
format="%d", value=job_details['Salary To'], key=f"salary_to_{job_id}")
            create_email = st.text_input("Email", value=job_details['Email'],
key=f"job_email_{job_id}")
```

```

        common_benefits = ['Health insurance', 'Dental insurance', 'Retirement
plan', 'Stock options', 'Paid time off', 'Remote work options', 'Gym membership',
'Free lunches', 'Childcare']
        selected_benefits = st.multiselect("Select benefits and perks",
common_benefits, key=f"selected_benefits_{job_id}")
        create_company_Size = st.number_input("Company Size", min_value=1,
step=1, format="%d", key=f"company_size_{job_id}")
        WorkType = ["Full-time", "Part-time", "Contract", "Casual"]
        WorkTypeSelection = st.selectbox("Work Type", WorkType,
index=WorkType.index(job_details['WorkType']), key=f"work_type_{job_id}")
        if st.button("Update Job", key=f"update_button_{job_id}"):
            collection.update_one(
                {"_id": ObjectId(job_id)},
                {"$set": {
                    "Job Category": create_JobCategory,
                    "Job Title": create_title,
                    "Job Description": create_Job_Description,
                    "Address line": create_location,
                    "City": create_city,
                    "State": create_state,
                    "Postcode": create_Postcode,
                    "WorkType": WorkTypeSelection, # Correct usage after defining
                    "Salary From": create_salary_from,
                    "Salary To": create_salary_to,
                    "Email": create_email,
                    "Benefit": selected_benefits,
                    "CompanySize": create_company_Size,
                }}
            )
            st.success("Job updated successfully!")
        else:
            st.error("No job found with the provided ID.")
        else:
            st.info("Please enter a MongoDB ID to update a job.")

```

Explanation:

The **update_job** function is a critical component of your job portal, enabling employers to modify existing job postings. When an employer inputs the MongoDB ID of a job, the system retrieves the job's current details. If the job is found, the employer can update various attributes such as job category, title, description, location, salary range, contact email, benefits, company size, and work type. These updates are made through a user-friendly interface that displays the current job details for editing. After making the desired changes, the employer can submit them, and the system updates the job details in the database using the `update_one` method. If the update is successful, the system confirms the changes to the employer. This feature is essential for maintaining the accuracy and relevance of job listings, adapting to changes in job roles or employer needs, and ensuring that job seekers have access to reliable information.

5.1.2.5 Delete Account and Associated Data

```
def delete_account():  
    st.header("Delete Account and All Related Jobs")  
  
    if 'username' in st.session_state and st.session_state['username']:  
        user_id = st.session_state['username']  
  
        confirm = st.text_input("Type 'DeleteAll' to confirm deletion",  
                                key="confirm_delete_account")  
  
        if st.button("Delete My Account"):  
            if confirm == 'DeleteAll':  
                job_result = collection.delete_many({"Business Name": user_id})  
                user_result = users_collection.delete_one({"username": user_id})  
  
                if user_result.deleted_count > 0:  
                    st.success("Your account has been deleted successfully.")  
                else:  
                    st.error("Failed to delete user account.")  
  
                clear_session_state()  
  
                st.info('You have been logged out. Your account and all associated jobs are  
deleted.')            else:  
                st.error("You must type 'DeleteAll' to delete your account and all jobs.")  
        else:  
            st.error("You are not logged in. Please log in to delete your account.")
```

Explanation:

Allows employers to delete all job listings they have posted. The user must type 'CONFIRM' to proceed with the deletion, which serves as a safeguard against unintended data loss. This functionality helps maintain a clean and relevant job listing database by enabling employers to efficiently manage their postings.

5.1.3 Job Listings

5.1.3.1 Overall Ranking Mechanism

```
if compare_button:
    query = build_query(job_category, state, work_type, salary_from, salary_to)
    filtered_jobs = list(collection.find(query))
    display_jobs(filtered_jobs, vectorizer)
```

Explanation:

- **query = build_query(job_category, state, work_type, salary_from, salary_to)**

which constructs a MongoDB query. It transforms user inputs into a query format that can be used to retrieve matching job listings from the database.

- **filtered_jobs = list(collection.find(query))**

After constructing the query, this line executes it against the MongoDB database to fetch job listings that match the specified criteria. The **collection.find(query)** function accesses the database, filters job entries according to the query parameters, and returns a list of jobs that meet these filters. This step is crucial as it sifts through potentially thousands of jobs to find those that align with the user's preferences, thereby streamlining the job search process.

- **display_jobs(filtered_jobs, vectorizer)**

This function takes the filtered job listings and processes them for display. each job description is vectorized using TF-IDF representation, allowing for the calculation of cosine similarities between the user's resume and each job description.

5.1.3.2 Query

```
def build_query(job_category, state, work_type, salary_from, salary_to):
    query = {}
    if job_category:
        query["Job Category"] = job_category
    if state:
        query["State"] = state
    if work_type:
        query["WorkType"] = {"$in": work_type}
    salary_from_value = 0 if salary_from == 'RM0' else
int(salary_from.strip('+K').strip('RM')) * 1000
    salary_to_value = int(salary_to.strip('+K').strip('RM')) * 1000 if salary_to !=
'50+K' else 100000
    query["Salary From"] = {"$gte": salary_from_value}
    query["Salary To"] = {"$lte": salary_to_value}
    return query
```

Explanation:

This function dynamically builds a MongoDB query based on user inputs. It checks if each filter (job category, state, work type, salary from, salary to) is set and adds the appropriate conditions to the query dictionary. The function handles the conversion of salary from a user-friendly format to a numerical format that can be used in MongoDB queries, enabling precise salary range filtering.

5.1.3.3 Ranking and Display

```
def display_jobs(filtered_jobs, vectorizer):
    def get_similarity_class(similarity):
        if similarity < 0.1:
            return 'low-similarity'
        elif similarity < 0.3:
            return 'moderate-similarity'
        else:
            return 'high-similarity'
    if 'resume_tfidf' in st.session_state and filtered_jobs:
        job_descs = [process_text(job['Job Description']) for job in filtered_jobs]
        job_descs_tfidf = vectorizer.transform(job_descs)
        resume_tfidf = st.session_state['resume_tfidf']

        # Calculate cosine similarity
        cos_similarities = cosine_similarity(resume_tfidf, job_descs_tfidf).flatten()
        for index, job in enumerate(filtered_jobs):
            job['Cosine Similarity'] = cos_similarities[index]

        # Sort jobs by cosine similarity
        sorted_jobs = sorted(filtered_jobs, key=lambda x: x['Cosine Similarity'],
reverse=True)
        for job in sorted_jobs:
            similarity_class = get_similarity_class(job['Cosine Similarity'])
            st.markdown(f"<div class='job-section'>", unsafe_allow_html=True)
            st.markdown(f"<h4>{job['Job Title']} at {job['Business Name']} (Similarity:
<span class='{similarity_class}'>{job['Cosine Similarity']:.2f}</span></h4>",
unsafe_allow_html=True)
            st.markdown(f"<div class='header'>Job Description:</div>",
unsafe_allow_html=True)
            with st.expander("See full description"):
                st.write(f"{job['Job Description']}")

            st.markdown(f"<div class='header'>Job Category:</div><div
class='content'>{job['Job Category']}</div>", unsafe_allow_html=True)
            st.markdown(f"<div class='header'>Location:</div><div
class='content'>{job['Address line']], {job['Postcode']] {job['City']],
{job['State']}</div>", unsafe_allow_html=True)
```

```

        st.markdown(f"<div class='header'>Work Type:</div><div
class='content'>{job['WorkType']}</div>", unsafe_allow_html=True)
        st.markdown(f"<div class='header'>Salary Range:</div><div
class='content'>{job['Salary From']} to {job['Salary To']}</div>",
unsafe_allow_html=True)
        st.markdown(f"<div class='header'>Email:</div><div class='content'><a
href='mailto:{job['Email']}'>{job['Email']}</a></div>", unsafe_allow_html=True)
        benefits = ", ".join(job.get('Benefit', [])) if job.get('Benefit', []) else "No
benefits listed"
        st.markdown(f"<div class='header'>Benefits and Perks:</div><div
class='content'>{benefits}</div>", unsafe_allow_html=True)
        st.markdown(f"<div class='header'>Company Size:</div><div
class='content'>{job.get('CompanySize', 'Not specified')}</div>",
unsafe_allow_html=True)
        st.markdown("</div>", unsafe_allow_html=True) # Close job-section div
        # st.write(f"**Contact Email:** {job['Email']}")
    else:
        st.markdown("<span class='no-info'>No resume uploaded or no jobs found
matching the criteria.</span>", unsafe_allow_html=True)

```

Explanation:

This function processes and displays the filtered job listings, ranking them based on their cosine similarity to the user's resume. It first converts job descriptions into TF-IDF vectors using the loaded vectorizer. Then, it calculates the cosine similarity between each job description and the user's resume. The jobs are sorted by this similarity score, and the details of each job are displayed in descending order of relevance. This method effectively prioritizes jobs that are most relevant to the user's skills, enhancing the job search experience by focusing on the most potentially suitable opportunities.

5.1.4 Visualization

5.1.4.1 Distribution of Job Listings by Job Category

```

def visualize_job_market():

    # Load and prepare data
    data = list(collection.find({}, {"Job Category": 1, "Salary From": 1, "Salary To": 1}))
    df = pd.DataFrame(data)
    df['Salary From'] = pd.to_numeric(df['Salary From'], errors='coerce')
    df['Salary To'] = pd.to_numeric(df['Salary To'], errors='coerce')
    df.dropna(inplace=True) # Clean the data

    # Compute average salary
    df['Average Salary'] = df[['Salary From', 'Salary To']].mean(axis=1)

```

```

summary_df = df.groupby('Job Category').agg({
    'Average Salary': 'mean',
    'Job Category': 'count'
}).rename(columns={'Job Category': 'Count'}).reset_index()

# Default to all categories for the pie chart
all_categories = summary_df['Job Category'].unique()
selected_categories = st.multiselect("Select Job Categories",
options=all_categories, default=all_categories)
filtered_data = summary_df[summary_df['Job
Category'].isin(selected_categories)]

# Allow user to choose which type of chart to display, default to Pie Chart
chart_type = st.selectbox("Choose Chart Type", ['Pie Chart', 'Bar Chart'],
index=0)

# Bar chart for average salary comparison
if chart_type == 'Bar Chart':
    fig_salary = px.bar(filtered_data, x='Job Category', y='Average Salary',
        title="Average Salary Comparison by Job Category",
        labels={"Average Salary": "Average Salary ($)"},
        color_discrete_sequence=px.colors.sequential.Pinkyl) # Pink-
themed color sequence
    fig_salary.update_traces(hovertemplate="<b>{x}</b><br>Average Salary:
${y:.2f}")
    st.plotly_chart(fig_salary)

# Pie chart for job listings distribution
elif chart_type == 'Pie Chart':
    fig_count = px.pie(filtered_data, names='Job Category', values='Count',
        title='Distribution of Job Listings by Job Category',
        color_discrete_sequence=px.colors.sequential.Agsunset) # Colorful
theme
    fig_count.update_traces(textinfo='percent+label',
        hovertemplate="<b>{label}</b><br>{value} listings
({percent})")
    st.plotly_chart(fig_count)

```

Explanation:

The "Distribution of Job Listings by Job Category" visualization effectively uses a pie chart to depict the proportional spread of job opportunities across various categories such as Engineering, Accounting, and Information Technology, providing a clear visual breakdown of the job market. This interactive visualization allows users to switch between a pie chart, which highlights the percentage distribution of jobs, and a bar chart, which emphasizes the average salary comparison by category. Such versatility in presentation caters to different user needs:

the pie chart offers a straightforward representation of job availability which aids job seekers in identifying fields with abundant opportunities, while the bar chart provides valuable salary insights useful for both job seekers and employers to understand economic aspects of the job market. The color-coded elements enhance readability and user engagement, allowing for an intuitive grasp of market trends. Additionally, users can interact with the charts by selecting specific job categories for detailed exploration, which makes this visualization a dynamic tool for strategic decision-making, be it for career planning or recruitment analysis. This dual-functionality not only enriches the user experience by providing tailored data insights but also turns the visualization into an essential feature for those looking to navigate the competitive landscape effectively.

5.2 Testing

In this Section, we explore the comprehensive testing framework developed to validate the functionality and reliability of the job-matching system. It includes test cases for the main user interface, employer interface, visualization features, and job listings. These test cases are designed to rigorously assess each component's performance under various scenarios, ensuring the system meets all specified requirements and functions correctly across different user interactions.

5.2.1 Main Page test Case

Test Case ID	Description	Preconditions	Steps	Expected Result	Actual Result
--------------	-------------	---------------	-------	-----------------	---------------

TC001	Test Resume Upload Functionality	User is on the main page	1) User click the upload button 2) User Select a resume file 3) User Confirms Upload	The resume is uploaded successfully and displayed	Successful
TC002	Test resume preprocessing	Resume is uploaded	1)System preprocesses the resume.	Resume is processed successfully	To be filled after testing
TC003	Verify the system can successfully extract phone numbers from uploaded resumes.	User has uploaded a resume with at least one valid phone number format.	1)Upload the resume. 2)System processes the resume and applies regex to extract phone numbers.	System extracts all listed phone numbers correctly.	Successful.
TF004	Verify the system can successfully extract email addresses from uploaded resumes.	User has uploaded a resume with at least one valid email address.	1)Upload the resume. 2)System Process the resume and applies regex to extract email addresses	System Extracts all listed email correctly	Successful
TC004	Verify the system's ability to extract names and language using NER	User upload a resume containing several names and languages	Upload the resume System Process the resume to identify and extract names and language	System identifies and extract all names and languages	Partially successful
TC005	Test TF-IDF Vectorization using Pre-saved Vectorizer	Resume has been successfully uploaded and preprocessed. A pre-saved TF-IDF vectorizer.	1)System retrieves the preprocessed text of the resume 2)System applies TF-IDF vectorization to transform the text data into a numerical vector	The text data is successfully converted into a TF-IDF vector format, ensuring compatibility with the prediction model.	Successful

TC006	Test Job Type Prediction	Resume text has been successfully vectorized.	1)System uses the TF-IDF vectorized data to predict the job type. 2) The prediction process leverages a pre-trained machine learning model.	The system correctly predicts the job type based on the vectorized resume.	Successful
TC007	Display Job Type Prediction	Job type has been predicted by the system.	1)System retrieves the predicted job type. 2)System displays the predicted job type to the user.	The predicted job type is correctly displayed on the user interface.	Successful

5.2.2 Employer Test Case

Test Case ID	Description	Preconditions	Steps	Expected Result	Actual Result
TC008	Test user registration functionality.	User is on the registration page.	1)User enter a new, unregistered user name. 2)User enters a password 3)User click the register button	System check if the user name is already registered. If not, the system encrypts the password using bcrypt, registers the user, and confirms registration success.	Successful
TC009	Test registration with an already registered username.	Username is already registered in the system.	1)User enters an already registered username. 2)User enters any password. 3) User clicks the register button.	System prevents registration and notifies the user that the username is already taken.	Error message indicating username is already taken.

TC010	Test login with correct credentials.	User's account is already registered.	1) User enters a registered username. 2) User enters the correct password. 3) User clicks the login button.	System checks credentials and logs the user in.	Successful login.
TC011	Test login with incorrect password.	User's account is registered but the wrong password is used.	1) User enters a registered username. 2) User enters an incorrect password. 3) User clicks the login button.	System denies access and notifies the user of incorrect credentials.	Error message indicating incorrect password.
TC012	Test logout functionality	User is logged into the system.	1) User clicks the logout button.	User is logged out and redirected to the login page	User is logged out successfully and sees the login page.
TC013	Test job creation with all required fields properly filled.	User is logged in and on the job creation page.	1) User fills in all mandatory fields correctly, including valid email and salary range. 2) User selects any optional benefits if desired. 3) User clicks the submit or create job button.	Job is created successfully and confirmation is shown.	Job is created and confirmation message displayed.
TC014	Test job creation with missing mandatory fields.	User is logged in and on the job creation page.	1) User leaves one or more mandatory fields (like Job Title or Email) empty. 2) User attempts to create a job by clicking the submit button.	System prevents job creation and displays error messages for all missing mandatory fields.	Error message indicating missing mandatory

					information.
TC015	Test job creation with an invalid email format.	User is logged in and on the job creation page.	1) User enters all required information but includes an invalid email format in the email field. 2) User attempts to create a job by clicking the submit button.	System prevents job creation and displays an error message regarding the invalid email.	Error message indicating invalid email format.
TC016	Test dynamic adjustment of 'Salary To' based on 'Salary From' value.	User is logged in and on the job creation page.	1) User enters a value in 'Salary From' (e.g., 1000). 2) User enters a lower value in 'Salary To' (e.g., 1). 3) User observes the automatic adjustment of the 'Salary To' field. 4) User tests the '-' and '+' buttons on 'Salary To' to decrease below or increase above the 'Salary From' value. 5) User sets 'Salary From' and 'Salary To' to the same value (e.g., 1000) and tries to decrease 'Salary To' using the '-' button.	1) When 'Salary To' is set lower than 'Salary From', it automatically updates to match the 'Salary From' value. 2) The '-' button is disabled when 'Salary From' and 'Salary To' are equal, preventing 'Salary To' from being less than 'Salary From'. 3) The '+' button allows increasing 'Salary To' above 'Salary From'.	Successful
TC017	Test reading job listings from the database.	User is logged in as an employer.	1) User navigates to the job listing page. 2) User clicks on the "Read Jobs" button.	→ If jobs associated with the user's company are present in the database, they are displayed. → If no jobs are associated, the message "No jobs found"	Successful

				for your company" is displayed.	
TC018	Test updating job details in the database.	User is logged in as an employer and navigates to the job update section.	1) User enters the MongoDB ID of the job they wish to update. 2) User modifies job details and submits the update. 3) If an incorrect ID format is entered, check for the error message.	→If the correct ID is provided, the job details are updated successfully in the database. →If an invalid ID format is entered, an "Invalid ID format. Please enter a valid MongoDB ObjectId."	Successful
TC019	Test deleting an individual job from the database.	User is logged in as an employer.	1) User enters the MongoDB ID of the job they wish to delete. 2) User clicks on the "Delete Job" button.	→If the correct ID is provided, the job is deleted successfully, and confirmation is displayed. →If an incorrect ID is entered, an "Invalid ID format. Please enter a valid MongoDB ObjectId." error message is displayed.	Successful
TC020	Test deleting all jobs associated with the employer's account.	User is logged in as an employer and navigates to the "Delete All Jobs" section.	1) User types "CONFIRM" in the input box to confirm deletion. 2) User clicks the "Delete All Jobs" button.	→If "CONFIRM" is correctly typed, all jobs associated with the user's account are deleted, and messages indicating the number of jobs found and deleted are displayed. →If "CONFIRM" is typed incorrectly (e.g., lowercase or incorrect spelling), the deletion does not proceed, and no jobs are deleted.	Successful

TC021	Test the deletion of an employer's account and all associated job postings.	User is logged in as an employer and navigates to the "Delete Account and All Related Jobs" section.	1) User types "DeleteAll" in the input box to confirm deletion. 2) User clicks the "Delete My Account" button.	→ If "DeleteAll" is correctly typed, the employer's account and all associated jobs are deleted successfully. Confirmation messages showing successful deletion of the account and jobs are displayed. The user is automatically logged out. → If "DeleteAll" is typed incorrectly (e.g., lowercase or incorrect spelling), the deletion does not proceed, and the account remains intact.	Successful
-------	---	--	---	---	------------

5.2.3 Job Listing Test Case

Test Case ID	Description	Preconditions	Steps	Expected Result	Actual Result
TC022	Test the job filtering functionality based on various criteria such as job category, state, type, and salary range.	User is on the Job listing page.	1) User selects various criteria → Job category → State → Work Type → Salary From and to (Ensuring Salary from is less than salary to) 2) User clicks the "Filter Jobs" button.	→ If all conditions are met and there are jobs matching the selected criteria, those jobs are displayed. → If Salary From is the same as or greater than Salary To, an error message is displayed. → If no jobs meet the selected criteria, a message "No jobs fulfill	Successful

				the selected requirements. “ is displayed	
TC023	Test the "Find Best Match" functionality to rank job listings based on cosine similarity when a resume is uploaded.	on the Job Listings page with available job listings.	1. Ensure no resume is uploaded and click the "Find Best Match" button. →Expected: An error message "No resume uploaded or no jobs found matching the criteria." is displayed. 2. Upload a resume and ensure no job matches the criteria and click the "Find Best Match" button. →Expected: The same error message is displayed as above. 3. Upload a resume, ensure jobs match the criteria, and click the "Find Best Match" button. →System calculates the cosine similarity between the resume and the job descriptions. →Jobs are ranked based on their cosine similarity scores and displayed in descending order. →Each job's similarity score is displayed alongside its details, with colors indicating the score range: ○ Below 0.1: Red color ○ 0.1 to 0.2: Blue color ○ Above 0.2: Green color	Jobs are correctly filtered, ranked, and displayed according to the cosine similarity with appropriate color coding for the similarity score. The listings are sorted in descending order based on the similarity score.	Successful

5.2.4 Visualization Interface Test Case

Test Case ID	Description	Preconditions	Steps	Expected Result	Actual Result
TC024	Test the functionality of switching between chart	User is on the Visuali	1) Select different job categories (e.g., Information	The graph should correctly display the data based on the selected job	Successful

	types and selecting job categories.	zation page.	Technology, Accounting, Engineering). 2) Switch between Pie Chart and Bar Chart options. 3) Observe changes in the graph display corresponding to the selections.	category and chart type. Pie chart shows distribution; bar chart shows comparative analysis.	
TC025	Test the filter functionality for job categories and multiple work types.	User is on the Visualization page.	1) Select a single job category. 2) Select multiple work types within the chosen category. 3) Observe the changes of the displayed graph.	The graph adjusts to display only the data that fits the selected job category and work types.	Successful
TC026	Test the comprehensive overview functionality, including the 'All' option in job category selections.	User is on the Visualization page.	1) Select individual job categories and observe the graph updates. 2) Select the 'All' option and verify that the graph provides a consolidated overview.	The graph should accurately reflect individual and combined data depending on the selection, with appropriate scaling and data aggregation.	Successful

5.3 Database and security Configuration

This chapter delves into the job-matching system's database management and security protocols. It discusses the utilization of two main MongoDB databases: the ResumeData database for storing job listings and the UserDetails database for managing user account information. The Section also highlights the security practices implemented, including bcrypt password hashing and network access configurations, to safeguard user data and ensure system integrity. Additionally, it outlines the rationale behind the current broad access settings and the planned security enhancements to tighten access controls as the system moves towards production.

5.3.1 MongoDB Usage

ResumeData Database: Primarily used for storing job listings and their details. This centralized repository facilitates the structured storage and retrieval of job data, which is crucial for the job matching and filtering functionalities of the system.

UserDetails Database: This database is dedicated to managing user account information, storing credentials including usernames and securely hashed passwords within the 'users' collection. This setup ensures organized and secure management of user authentication data.

5.3.2 Security Practices

Password Hashing: For enhanced security, bcrypt hashing is employed to safeguard user passwords. By hashing passwords before storage in the UserDetails database, we ensure they are never kept in plain text, thereby increasing protection against unauthorized access and potential data breaches.

5.3.3 Network and Server Settings

Access Configuration: The MongoDB instance is currently set to allow connections from any IP address to accommodate various stages of development and testing. This setting ensures ease of access for system testing and demonstration purposes from different geographic locations.

Rationale for Broad Access: The decision to allow broad access stems from the current developmental phase of the project, which lacks a defined user base and is primarily focused on functionality testing and demonstrations. This configuration helps avoid potential connectivity issues or access restrictions that could impede these activities.

Future Security Enhancements: As the project progresses towards a production environment and begins to handle sensitive user data, it will be crucial to implement stricter network security measures such as IP whitelisting. This change will restrict database access to authorized locations only, substantially improving security and preventing unauthorized access.

Chapter 6 *(if applicable)*

System Deployment

6 System Deployment

6.1 Local Deployment Instructions

To run this project locally, follow these steps to ensure proper setup and execution:

Prerequisites:

Ensure Python and Streamlit are installed on your system. You can install Streamlit using pip if it's not already installed:

```
pip install streamlit
```

Clone or Download the Project:

Obtain a copy of the project files from the source. If provided via a repository, you can clone it using Git or download the ZIP file and extract it on your local machine.

Update File Paths:

Before running the application, update the paths for the following files in the code to reflect their locations on your system:

- **Vectorizer file:** Update the path in the application where the vectorizer is loaded (e.g., `vectorizer_path = '<YourPath>/vectorizer.pkl'`).
- **Machine Learning Model file:** Update the path in the application where the logistic regression model is loaded (e.g., `model_filename = '<YourPath>/tuned_logistic_regression_model.pkl'`).

Running the Application:

- Open a terminal or command prompt.
- Navigate to the directory containing the project files.
- Run the application by executing the following command:

```
streamlit run <PathToFile>/fyp1.py
```

- Replace `<PathToFile>` with the actual path to the `fyp1.py` file on your local machine.

Access the Application:

Once the application is running, open a web browser and visit <http://localhost:8501> to interact with the application.

Chapter 7

Discussions and Conclusion

7 Discussions and Conclusion

This section of the report provides a comprehensive evaluation of the project, detailing the challenges encountered, the solutions implemented, and areas for future improvement. The discussion aims to highlight the project's strengths and limitations, providing a clear perspective on its practical implications and potential marketability.

7.1 Summary

This project addresses the challenges in the job recruitment process, particularly the inefficiencies caused by handling a high volume of applications and the prevalence of unclear job descriptions. By leveraging advanced algorithms like cosine similarity and TF-IDF, the proposed solution aims to streamline the application screening process and improve the match between job descriptions and applicant profiles. This approach is designed to reduce mismatched applications, decrease hiring costs, and shorten the hiring cycle. Additionally, the use of automated screening is intended to minimize unconscious biases, thereby promoting a more diverse and inclusive workplace. The justification for choosing specific tools and methodologies, such as cosine similarity and TF-IDF, stems from their proven effectiveness in text analysis and their capability to enhance the speed and accuracy of matching candidates to job descriptions. This project not only tackles the core inefficiencies found in traditional recruitment processes but also enhances the overall experience and outcomes for all stakeholders involved in hiring.

7.2 Achievements

In evaluating the project against its objectives, the system has notably advanced job matching capabilities by leveraging NLP techniques such as TF-IDF vectorization and cosine similarity algorithms, enhancing the precision of matches between job seekers and job listings. This automation reduces the manual workload on HR departments, streamlining the hiring process and effectively managing large volumes of applications. Despite its strengths in technological innovation and a user-centric design that improves job seeker engagement, the project presents certain weaknesses: it offers limited tools for employers and relies on static data, which can hinder its adaptability to real-time market changes and potentially lead to outdated job matches. Additionally, while the system reduces hiring biases by focusing on qualifications and minimizing subjective criteria, it requires more advanced security protocols to ensure robust data protection. The architecture's scalability is a significant advantage, yet the complexity of maintaining and updating AI algorithms poses challenges for long-term deployment. To

enhance its marketability and functionality, future improvements should focus on developing dynamic, real-time data processing capabilities, enriching employer-centric features, and revamping the user interface to make it more intuitive and appealing. These enhancements would ensure the system remains competitive and effective in the evolving job market, making it a valuable tool for both job seekers and employers.

7.3 Contributions

The proposed system offers a transformative approach to streamlining recruitment processes, employing advanced natural language processing techniques like TF-IDF vectorization and cosine similarity to precisely match job seekers with optimal employment opportunities. This system is particularly necessary in today's fast-paced job market where efficiency and precision in hiring are crucial. By reducing the high volume of mismatched applications, the system not only decreases recruitment costs and timescales but also enhances the quality of matches between employers and job candidates. Its marketability extends beyond traditional job platforms; it is especially valuable for universities helping students secure internships and for job-hunting websites seeking to optimize their service offerings. Furthermore, by automating the initial screening processes, the system helps to minimize unconscious hiring biases, promoting a more diverse and inclusive workforce. This positions the system as an innovative tool that can revolutionize recruitment processes in educational institutions and commercial job markets alike, making it a marketable solution to improve how companies, students, and educational bodies connect with potential job opportunities.

7.4 Limitations and Future Improvements

Employer-Centric Features: Currently, the system is heavily optimized for job seekers, focusing on matching them with suitable jobs based on their resumes. This can be expanded to include more employer-centric features, such as tools for analyzing the pool of applicants in terms of skills distribution and job fit, which would aid employers in making more informed hiring decisions.

Broader Range of Job Criteria: The matching algorithm primarily considers job descriptions and resume content. This could be expanded to include other important factors such as cultural fit, expected job tenure, and other soft skills that are often crucial for successful employment but difficult to quantify.

Enhanced Security Features: While the system uses bcrypt for securing passwords, overall data security could be enhanced. This includes implementing more robust data encryption

methods during data transmission and at rest, and introducing more strict access controls and audit trails to monitor data access and manipulation.

Bias Mitigation: Although the system aims to minimize unconscious biases by focusing on qualifications and experience, there is still a potential for bias in the job descriptions themselves or in the way algorithms interpret data. Future improvements could include implementing AI fairness tools and techniques to detect and mitigate potential biases in job matching.

Scalability and Performance: As the system scales, maintaining performance will become increasingly critical. Future developments could focus on optimizing the backend architecture to handle a larger number of simultaneous users and more complex queries without degradation in system response times.

User Interface and Experience: Enhancing the user interface to make it more intuitive and user-friendly can significantly improve user satisfaction. Adding interactive elements, personalized dashboards, and more customizable job search options could make the system more engaging and easier to use.

7.5 Issues and Solutions

During the development of this project, several challenges arose, predominantly technical in nature, such as ensuring the accuracy and efficiency of the matching algorithms (TF-IDF and cosine similarity). To tackle these, extensive testing and tuning were conducted, which improved the system's capability to effectively match job descriptions with candidate profiles. Additionally, managing the integration of various technologies like MongoDB, Streamlit, and the NLP tools required careful coordination. This was addressed by modular development and incremental testing, which helped isolate and resolve issues systematically.

Project management also posed a challenge due to the complexity of integrating multiple components. Effective time management and prioritization were crucial in overcoming these obstacles. Regular review sessions were instituted to track progress and realign goals, ensuring the project stayed on schedule.

One of the more profound lessons learned was the importance of data quality in training the models. Initial results were suboptimal due to inconsistent data, which was mitigated by refining data preprocessing steps to ensure cleaner, more uniform inputs for the machine learning models.

The experience also highlighted the potential for user feedback to refine system functionality. In the future, incorporating user testing phases could provide insights that lead to more intuitive interfaces and features, enhancing overall usability.

These challenges and their resolutions not only ensured the project's completion on time but also enriched the learning experience, providing practical insights into managing a multifaceted software development project.

References

- A Comprehensive Guide to Named Entity Recognition (NER)*. (2022, June 10). <https://www.turing.com/kb/a-comprehensive-guide-to-named-entity-recognition>
- Achsan, B. M. (2021, December 12). Support Vector machine: Regression - IT Paragon – Medium. *Medium*. <https://medium.com/it-paragon/support-vector-machine-regression-cf65348b6345>
- A History of Recruitment - Era 2: the 1980s to 1990* | *EMInfo.com*. (n.d.). *EMInfo.com*. <https://www.eminfo.com/articles-a-history-of-recruitment--era-2-the-1980s-to-1990-1013.php>
- Abid Ali, A. (2023, September). What is Tokenization? *Datacamp*. <https://www.datacamp.com/blog/what-is-tokenization>
- Agata. (2023, December 13). What is named entity recognition and how does it work? *Klippa*. <https://www.klippa.com/en/blog/information/named-entity-recognition/>
- Alabi, T. H. (2022, November 3). Python lower() – How to Lowercase a Python String with the tolower Function Equivalent. *freeCodeCamp.org*. <https://www.freecodecamp.org/news/how-to-lowercase-python-string/>
- Anirudha Simha, Principle Associate Software Engineer, Kai Chatbot Team. (2021, October 6). *Understanding TF-IDF for Machine Learning*. Capital One. <https://www.capitalone.com/tech/machine-learning/understanding-tf-idf/>
- Antoniadis, P., & Antoniadis, P. (2023, June 17). What is cosine similarity? | *Baeldung on Computer science*. *Baeldung on Computer Science*. <https://www.baeldung.com/cs/cosine-similarity>
- Anushka, A., & Senthilkumar. (2022). RESUME RECOMMENDATION SYSTEM USING COSINE SIMILARITY. *IRJMETs*, 04(04), e-ISSN: 2582-5208.
- Arya, N. (n.d.). *TF-IDF defined* - *KDNuggets*. *KDNuggets*. <https://www.kdnuggets.com/2022/10/tfidf-defined.html>
- Chaudhary, M. (2021, December 14). *TF-IDF Vectorizer scikit-learn* - *Mukesh Chaudhary* - *Medium*. *Medium*. <https://medium.com/@cmukesh8688/tf-idf-vectorizer-scikit-learn-dbc0244a911a>
- Chen, K. (2022, January 6). Introduction to Natural Language Processing — TF-IDF. *Medium*. <https://kinderchen.medium.com/introduction-to-natural-language-processing-tf-idf-1507e907c19>
- Chouksey, S. (2021, December 14). Demonstrating calculation of TF-IDF from Sklearn - *Analytics Vidhya* – *Medium*. *Medium*. <https://medium.com/analytics-vidhya/demonstrating-calculation-of-tf-idf-from-sklearn-4f9526e7e78b>
- Deepanshi. (2024, April 24). *Text Preprocessing in NLP with Python Codes*. *Analytics Vidhya*. <https://www.analyticsvidhya.com/blog/2021/06/text-preprocessing-in-nlp-with-python-codes/>
- Duque, T. (2021, December 13). How to build a Lemmatizer - *Analytics Vidhya* - *Medium*. *Medium*. <https://medium.com/analytics-vidhya/how-to-build-a-lemmatizer-7aeff7a1208c>
- Educative. (n.d.). How to remove stop words using spaCy in Python. <https://www.educative.io/answers/how-to-remove-stop-words-using-spacy-in-python>
- Fares, S. (2022, September 23). *Text similarity with word embeddings in Gensim*. *Medium*. <https://medium.com/nlplanet/text-similarity-with-the-next-generation-of-word-embeddings-in-gensim-466fdafa442>
- Fatih, K. (n.d.). *Cosine similarity*. <https://www.learndatasci.com/glossary/cosine-similarity/>

- Foong, N. W. (2021, December 10). The Beginner's guide to similarity matching using SPACY. *Medium*.
<https://betterprogramming.pub/the-beginners-guide-to-similarity-matching-using-spacy-782fc2922f7c>
- GeeksforGeeks. (2017, May 23). Tokenize text using NLTK in python. *GeeksforGeeks*.
<https://www.geeksforgeeks.org/tokenize-text-using-nltk-python/>
- GeeksforGeeks. (2022, September 16). Tokenization Using Spacy library. *GeeksforGeeks*.
<https://www.geeksforgeeks.org/tokenization-using-spacy-library/>
- GeeksforGeeks. (2023, January 11). Python Measure similarity between two sentences using cosine similarity. *GeeksforGeeks*.
<https://www.geeksforgeeks.org/python-measure-similarity-between-two-sentences-using-cosine-similarity/>
- GeeksforGeeks. (2023, July 28). Remove Special Characters from String Python. *GeeksforGeeks*.
<https://www.geeksforgeeks.org/python-removing-unwanted-characters-from-string/>
- GeeksforGeeks. (2024, January 3). Introduction to stemming. *GeeksforGeeks*.
<https://www.geeksforgeeks.org/introduction-to-stemming/>
- GeeksforGeeks. (2024, July 4). Support Vector Machine (SVM) algorithm. *GeeksforGeeks*.
<https://www.geeksforgeeks.org/support-vector-machine-algorithm/>
- Gillis, A. S., & Botelho, B. (2023, March 7). MongoDB. Data Management.
<https://www.techtarget.com/searchdatamanagement/definition/MongoB>
- Global, C. (2022, August 18). What are the major challenges faced by job seekers in conducting an effective job search?
<https://www.linkedin.com/pulse/what-major-challenges-faced-job-seekers-conducting-effective->
- Hiredna. (2023, February 15). 60% of Candidates Ditch the Application Because it's Too Long or Complex: How to Simplify the Job Application Process. *HireDNA*.
<https://hiredna.com/60-of-candidates-ditch-the-application-because-its-too-long-or-complex-how-to-simplify-the-job-application-process/>
- Jagwani, V., Meghani, S., Pai, K., & Dhage, S. (2023, July 28). Resume Evaluation through Latent Dirichlet Allocation and Natural Language Processing for Effective Candidate Selection. *arXiv.org*.
<https://arxiv.org/abs/2307.15752>
- Katarina, S. (2023, July 7). Understanding Cosine Similarity in Python with Scikit-Learn. *Memgraph*.
<https://memgraph.com/blog/cosine-similarity-python-scikit-learn>
- Kotamraju, S. (2022, June 11). How to use BERT from the Hugging Face transformer library. *Medium*.
<https://towardsdatascience.com/how-to-use-bert-from-the-hugging-face-transformer-library-d373a22b0209>
- Kfollis. (2023, October 31). Microsoft Logistic Regression Algorithm. *Microsoft Learn*.
<https://learn.microsoft.com/en-us/analysis-services/data-mining/microsoft-logistic-regression-algorithm?view=asallproducts-allversions>
- Lora, K. (2023, February 12). How long does it take to get hired? *Linkedin*.
<https://www.linkedin.com/pulse/how-long-does-it-take-to-get-hired-get-hired-by-linkedin-news>

Marcelo, S. (2020, September 17). *An introduction to unconscious bias – from industry experts*. BrightTALK.

<https://www.brighttalk.com/webcast/17947/438643>

Marshall, C. (2021, December 13). What is named entity recognition (NER) and how can I use it? *Medium*.

<https://medium.com/mysupera/what-is-named-entity-recognition-ner-and-how-can-i-use-it-2b68cf6f545d>

Mishra, A. (2022, January 4). How to change string data or text data of a column to lowercase in Pandas? *Medium*.

<https://aparnamishra144.medium.com/how-to-change-string-data-or-text-data-of-a-column-to-lowercase-in-pandas-248a8ce4ae01>

MongoDB. (n.d.). *Schemaless database*.

<https://www.mongodb.com/resources/basics/unstructured-data/schemaless>

Nabil, M. (2023, February 20). Text normalization (e.g., stemming, lemmatization, lowercasing). *Medium*.

<https://medium.com/@madali.nabil97/text-normalization-e-g-stemming-lemmatization-lowercasing-833558a3ca21>

Navarra, K. (2023, December 21). The real costs of recruitment. *SHRM*. <https://www.shrm.org/topics-tools/news/talent-acquisition/real-costs-recruitment>

`pandas.Series.str.replace` — *pandas 2.2.2 documentation*.

(n.d.). <https://pandas.pydata.org/docs/reference/api/pandas.Series.str.replace.html>

Prabhakaran, S. (2022, April 20). Cosine Similarity – Understanding the math and how it works (with python codes). *Machine Learning Plus*. <https://www.machinelearningplus.com/nlp/cosine-similarity/>

Prabhakaran, S. (2022, April 20). Cosine Similarity – Understanding the math and how it works (with python codes). *Machine Learning Plus*. <https://www.machinelearningplus.com/nlp/cosine-similarity/>

Pradeep. (2024, February 11). Understanding TF-IDF in NLP: A Comprehensive Guide - Pradeep - *Medium*.

Medium. <https://medium.com/@er.iit.pradeep09/understanding-tf-idf-in-nlp-a-comprehensive-guide-26707db0cec5>

Reporter, G. S. (2020, July 1). “Vague” job ads waste applicants’ and employers’ time – Citizens Advice. *The Guardian*.

<https://www.theguardian.com/money/2015/jun/17/vague-job-ads-waste-applicants-and-employers-time-citizens-advice>

Roy, P. K., Chowdhary, S. S., & Bhatia, R. (2020b). A Machine Learning approach for automation of Resume

Recommendation system. *Procedia Computer Science*, 167, 2318–2327. <https://doi.org/10.1016/j.procs.2020.03.284>

Sarguroh, F. (2023, July 10). 03: Basic Text Preprocessing (NLP) - Farhan Sarguroh - *Medium*.

<https://farhansarguroh.medium.com/03-basic-text-preprocessing-nlp-90ea4469fb87>

Schulze, J. (2024, January 29). What is Named Entity Recognition (NER) and how does it work? *Coursera*.

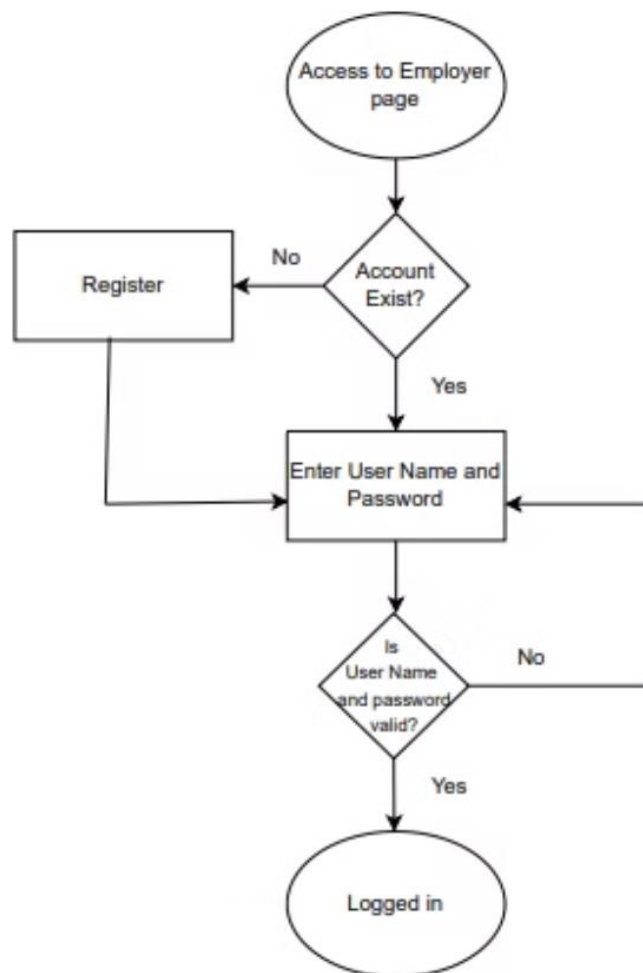
<https://www.coursera.org/articles/named-entity-recognition>

Sharma, K. (2021, February 4). What is Elasticsearch and how are enterprises using it? *Ubuntu*.

<https://ubuntu.com/blog/what-is-elasticsearch>

- Solutions, I. H. E. (2024, March 27). *Unlocking Success: How the digital connected world drives business growth, talent acquisition, and professional networking in the Post-Pandemic era.* https://www.linkedin.com/pulse/unlocking-success-how-digital-connected-zrire?trk=public_post_main-feed-card_feed-article-content
- SourceBreaker, & SourceBreaker. (2023, February 16). *Traditional Recruitment vs. Modern Recruitment.* SourceBreaker. <https://www.sourcebreaker.com/traditional-vs-modern-recruitment/>
- Simplilearn. (2023, November 7). *Random Forest Algorithm.* Simplilearn.com. <https://www.simplilearn.com/tutorials/machine-learning-tutorial/random-forest-algorithm>
- Team, G. (2018, December 21). *50 HR & Recruiting Stats that make you think|50 HR & recruiting stats that make you think.* US | Glassdoor for Employers. <https://www.glassdoor.com/employers/blog/50-hr-recruiting-stats-make-think/>
- Tijare, P., Waseem, M., Sherani, M. A., Krishna, K. S. K. S., & Kavitha, P. (2023). *A smart resume screening tool for customized shortlisting.* ITM Web of Conferences, 56, 04001. <https://doi.org/10.1051/itmconf/20235604001>
- UpJourney. (2023, March 31). *Why are job descriptions important in the recruiting process?* <https://upjourney.com/why-are-job-descriptions-important-in-the-recruiting-process>
- What is named entity recognition? | IBM.* (n.d.). <https://www.ibm.com/topics/named-entity-recognition>
- What is Random Forest? | IBM.* (n.d.-b). <https://www.ibm.com/topics/randomforest#:~:text=Random%20forest%20is%20a%20commonly,both%20classification%20and%20regression%20problems.>
- W3Schools.com. (n.d.). https://www.w3schools.com/python/ref_string_replace.asp
- W3Schools.com. (n.d.-b). https://www.w3schools.com/python/ref_string_split.asp
- Yasar, K., Lawton, G., & Burns, E. (2024, April 25). *logistic regression.* Business Analytics. <https://www.techtarget.com/searchbusinessanalytics/definition/logistic-regression#:~:text=Logistic%20regression%20also%20comes%20with,Overfitting%20and%20sensitivity%20to%20outliers.>
- Zuker, A. (2023, November 23). *Two-Decade Journey Through the Evolution of HRMS/HRIS with a glimpse into the future.* <https://www.linkedin.com/pulse/two-decade-journey-through-evolution-hrms-hris-glimpse-amir-zuker-strlc>

Appendices



Algorithm Of login and register account.

DataBase Account and password.

DataBase → MongoDB

account : ongkaifyp@gmail.com

password:

DataBase to store UserName and password

The screenshot shows the MongoDB Atlas interface for a Cluster named 'KAI'S ORG - 2024-09-05 > PROJECT 0 > DATABASES'. The 'Collections' tab is selected, showing the 'user_details.users' collection. The collection has a storage size of 36KB, logical data size of 435B, total documents of 4, and index total size of 36KB. The 'Find' tab is active, displaying a query filter: `{ field: 'value' }`. The query results show 1-3 of 3 documents. The first document is:

```
{
  "_id": ObjectId("66e89dad486dd7c10b2167be"),
  "username": "AtrAsia",
  "password": Binary.createFromBase64("3D313DEy3HRBQmh1QzZvbTbWtSSyMnRWV1NmMvYXNkLzZcQ1L2RmWTQaFRUVjBRTJZHRXNHln3yVj8x", ...)
}
```

DataBase to store Job and Job details

The screenshot shows the MongoDB Atlas interface for the same Cluster. The 'Collections' tab is selected, showing the 'Resume.ResumeData' collection. The collection has a storage size of 62KB, logical data size of 23.9KB, total documents of 15, and index total size of 36KB. The 'Find' tab is active, displaying a query filter: `{ field: 'value' }`. The query results show 1-15 of 15 documents. The first document is:

```
{
  "_id": ObjectId("66e8abed063b02e88ccd14c8"),
  "Business Name": "AtrAsia",
  "Job Category": "Information-Technology",
  "Job Title": "Senior Data Scientist",
  "Job Description": "We are seeking a Senior Data Scientist to design, develop, and deploy ...",
  "Address Line": "...",
  "City": "Kuala Lumpur"
}
```

This page is intentionally left blank to indicate the back cover. Ensure that the back cover is black in color.